AI505

Optimization

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

List of Lectures

1. Introduction
2. Derivatives and Gradients
3. Bracketing
4. Local Descent
5. First-Order Methods
6. Second-Order Methods

1. Introduction

Who is here?

44 in total registered in ItsLearning

- **AI505 (7.5 ECTS)**
  31 from Bachelor in AI

- **AI801 (10 ECTS)** or **AI505 (7.5 ECTS)** + **IAAI501 (2.5 ECTS)**
  13 from Master Educations (which ones?)

Prerequisites

- Calculus
- Linear Algebra
- Programming

Outline

1. Course Organization

2. Introduction

Aims of the course

Learn about **optimization**:

- continuous multivariate optimization
- discrete optimization

Optimization is an important tool in **machine learning**, **decision making** and in analysing physical systems.
In mathematical terms, an **optimization problem** is the problem of finding the **best solution** from the set of all **feasible solutions**.

The first step in the optimization process is constructing an appropriate **mathematical formulation**. The second is devising an **algorithm** for solving the mathematical formulation.

Contents of the Course (Pensum)

Unit	Main topic
1	Introduction, Univariate Problems
2	Multivariate Problems, Gradient-Based Methods
3	Derivative-Free Methods
4	Optimization for Machine Learning
5	Constrained Optimization, Linear Programming
6	Sampling Methods
7	Discrete Optimization and Heuristics

Practical Information

Teacher: Marco Chiarandini (imada.sdu.dk/u/marco/)
Instructor: Sai Ganesh Nagarajan (H21)

Schedule, alternative views:

- mitsdu.sdu.dk, SDU Mobile
- Official course description (skemaplanen)
- ItsLearning
- https://ai-505.github.io

Schedule (16 weeks):

- Introductory classes: 16 classes (32 hours)
- Training classes: 16 classes (32 hours)
- Scheduled: $(16 \times 2) + 2$ classes $\times 2 = 68$ hours

Communication Means

- ItsLearning $\Leftrightarrow$ External Web Page
  (link `https://ai-505.github.io`)
- Announcements + Discussion Board + Submissions in ItsLearning
- Write to Marco (`marco@imada.sdu.dk`) or to instructor
- Collaborate with peers

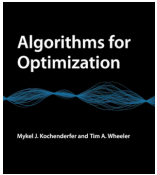
$\rightsquigarrow$ **It is good to ask questions!!**

$\rightsquigarrow$ **Let me know if you think we should do things differently!**

Sources — Reading Material

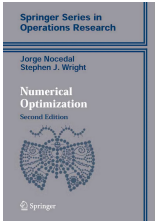
Main reference:

[KW] Mykel J. Kochenderfer, Tim A. Wheeler. Algorithms for Optimization 2019. The MIT Press.



Others

[NW] J. Nocedal and S. J. Wright, Numerical Optimization, Second Edition. Springer Series in Operations Research, 2006



Course Material

External Web Page is the main reference for list of contents (ie[1], syllabus, pensum).

It will collect:

- slides
- list of topics and references
- exercises
- links
- tutorials for programming tasks

[1] ie = id est = that is; eg = exempli gratia = for example; wrt = with respect to; et al. = et alii = and others

Assessment

Portfolio consisting of:

- mandatory assignments in groups of 2:
  1. assignment
  2. assignment

  Time line to be announced.

- oral exam on June 29-30, 2026

The oral examination consists of questions on the basis of the handed in assignments and can be extended to other parts of the course curriculum.

Both elements must individually meet the learning objectives.

Final grade: primarily based on element 1, but element 2 may adjust the grade up or down by one grade level.

(language: Danish and/or English)

Exercise Sessions (1/2)

For exercises in general:

- Both theoretical, modelling and programming tasks
- No mandatory hand-ins, voluntary participation, but all lectures and exercises are relevant for the exam
- Help each other! Teaching others is the best form of learning
- Sai will be there to help. First hour: you work and Sai gives personalize help. Second hour: Sai presents a subset of solutions in plenary.
- Exercises should help you to learn - find the setting that works best for you, don't hesitate to extend with other material that works better for you. You can use Gen AI tools.

Exercises are group work:

- Exercises best done in pairs of 2-3 people
- Try to gather in different groups every now and then

Exercise Sessions (1/2)

Notation: $^{+}$, *, ' '

- plus exercises are to be done before the class
- starred exercises are done in class. They are good examples of assignment questions
- unmarked exercises are for self study

Coding

Set up your local Python programming environment and use the available tutorial to:

- Brush up some knowledge on Python-IDEs and Git
- Grasp some basics on jupyter notebooks
- Recap basics of data processing and visualization

We will span several programming modes: functional, imperative, object oriented, small scripts, large code bases, use of modules and libraries. We will use git.

Outline

1. Course Organization

2. Introduction

Introduction

- Applications of Optimization
  - Physics
  - Business
  - Biology
  - Engineering
  - Machine Learning
  - Logistics and Scheduling
- Objectives to Optimize
  - Efficiency
  - Safety
  - Accuracy
- Constraints
  - Cost
  - Weight
  - Structural Integrity
- Challenges
  - High-Dimensional Search Spaces
  - Multiple Competing Objectives
  - Model Uncertainty

Real World, Model, Representation, Implementation

The Real World: That messy thing we are trying to study (with computers).

Model: Mathematical object in some class M.

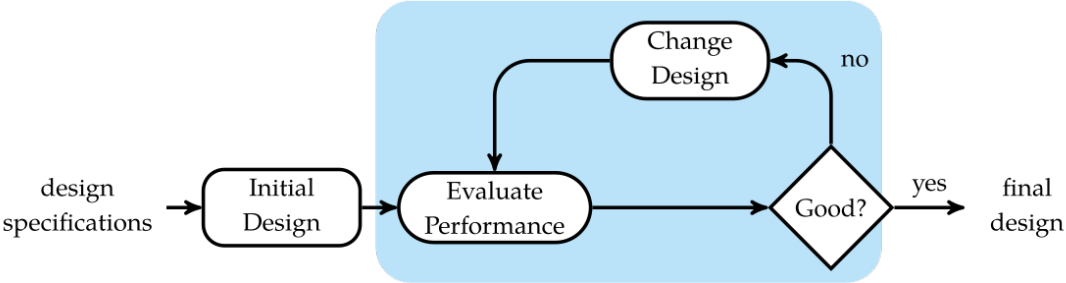
Representation: An object of an abstract data type R used to store the model.

Implementation: An object of a concrete type used to store the model.

Any object from the real world might have different models.
Any model might have several representations (exact).
And representation might have different implementations (exact).

We will focus on the algorithmic aspects of optimization that arise after the problem has been properly formulated

Optimization Process



An optimization algorithm is used to incrementally improve the design until it can no longer be improved or until the budgeted time or cost has been reached.

Optimization Process

Search the space of possible designs with the aim of finding the best one.

Depending on the application, this search may involve:

- evaluating an analytical expression (white or glass box)
- running physical experiments, such as wind tunnel tests (black box)
- running computer simulations

Modern optimization techniques can be applied to problems with millions of variables and constraints.

Definitions – Functions

- Univariate single-valued function: $f : \mathbb{R} \to \mathbb{R}$ is a function that takes as input a single real number, say x, and produces a single real number as output.
- Multivariate single-valued function: $f : \mathbb{R}^n \to \mathbb{R}$ is a function that takes as input a vector $\boldsymbol{x}$ of n real numbers and produces a single real number as output.
- (Multivariate) Vector-valued function: $f : \mathbb{R}^n \to \mathbb{R}^m$ is a function that takes as input a vector $\boldsymbol{x}$ of n real numbers and produces a vector $\boldsymbol{y}$ of m real numbers as output.

We will work primarily with multivariate single-valued functions.

Basic Optimization Problem

$$\begin{aligned} &\underset{\boldsymbol{x}}{\text{minimize}}\ f(\boldsymbol{x}) \\ &\text{subject to } \boldsymbol{x} \in \mathcal{X} \end{aligned}$$

- Feasible Set $\mathcal{X} \subseteq \mathbb{R}^n$
- Design Point $\boldsymbol{x}$
- Design Variables
- Objective Function: $f : \mathbb{R}^n \to \mathbb{R}$ (scalar-valued function)
- Minimizer

Any value of $\boldsymbol{x}$ from among all points in the feasible set $\mathcal{X}$ that minimizes the objective function is called a **solution** or **minimizer**. A particular solution is written $\boldsymbol{x}^*$.

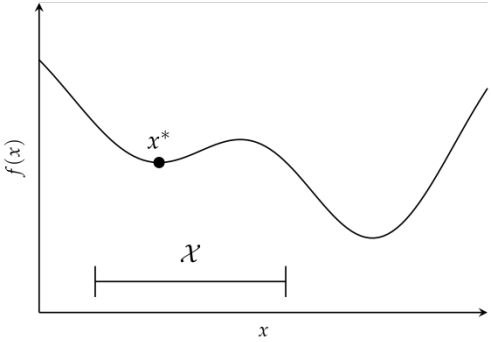
$$\boldsymbol{x}^* = \text{argmin}\, f(\boldsymbol{x}) \text{ subject to } \boldsymbol{x} \in \mathcal{X}$$

There is only one minimum but there can be many minimizers

maximize $f(\boldsymbol{x})$ subject to $\boldsymbol{x} \in \mathcal{X} \equiv$ minimize $-f(\boldsymbol{x})$ subject to $\boldsymbol{x} \in \mathcal{X}$

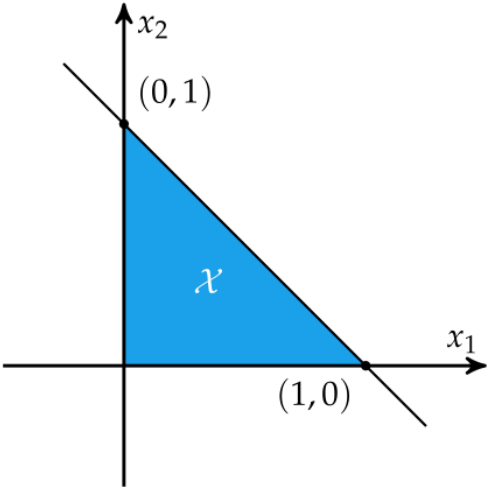
Basic Optimization Problem

$$\begin{aligned} \underset{\boldsymbol{x}}{\text{minimize}} \quad & f(\boldsymbol{x}) \\ \text{subject to} \quad & \boldsymbol{x} \in \mathcal{X} \end{aligned}$$



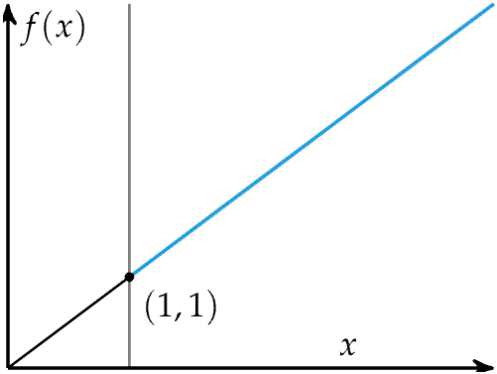
Constraints

$$\begin{aligned} \underset{x_1, x_2}{\text{minimize}} \quad & f(x_1, x_2) \\ \text{subject to} \quad & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1 \end{aligned}$$



Constraints

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & f(x) \\ \text{subject to} \quad & x > 1 \end{aligned}$$



The problem has no solution.
$x = 1$ would not be feasible.

$x = 1$ would be the solutions to

$$\underset{x}{\text{infimum}}\ f(x) \text{ subject to } x > 1$$

infimum of a subset $\mathcal{X}$ of a partially ordered set $\mathcal{P}$ is the greatest element in P that is less than or equal to each element of $\mathcal{X}$, if such an element exists. Aka, **greatest lower bound**.

Decision Space and Objective Space

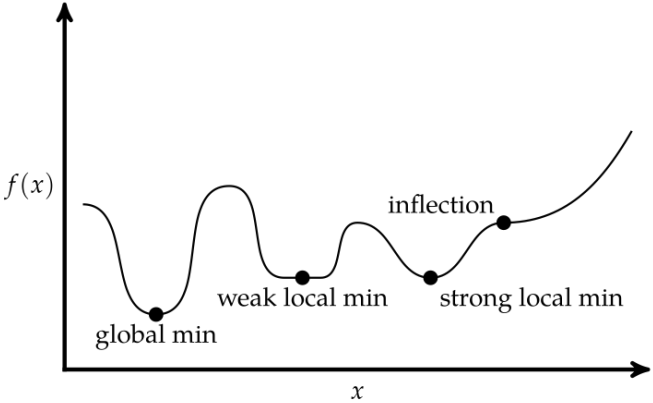
Plot to add.

Critical Points

Univariate function

global vs local minimum

<u>Def.</u> A point $\boldsymbol{x}^*$ is at a **local minimum** (or is a local minimizer) if there exists a $\delta > 0$ such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$ with $\|\boldsymbol{x} - \boldsymbol{x}^*\| < \delta$.

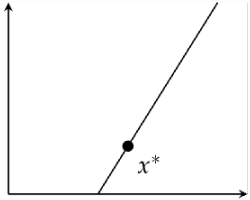


Conditions for Local Minima

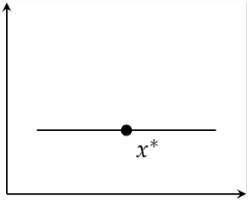
Univariate objective functions, assuming they are differentiable (Derivatives exist), without constraints

Local minimum: Necessary condition but not sufficient condition:

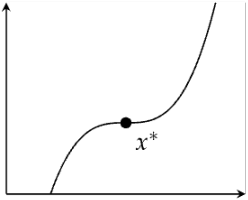
1. $f'(x^*) = 0$, the first-order necessary condition (FONC)
2. $f''(x^*) \geq 0$, the second-order necessary condition (SONC)



SONC but not FONC



FONC and SONC



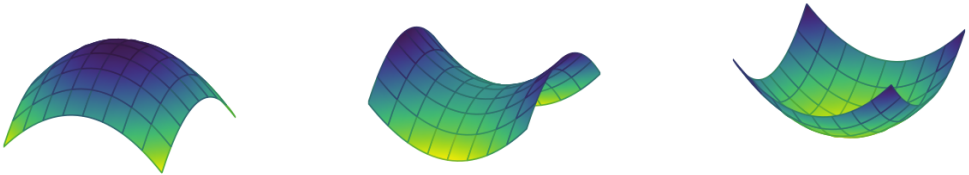
FONC and SONC

Conditions for Local Minima

Multivariate objective functions, assuming they are differentiable (gradients and Hessians exist), without constraints

Local minimum: Necessary condition but not sufficient condition:

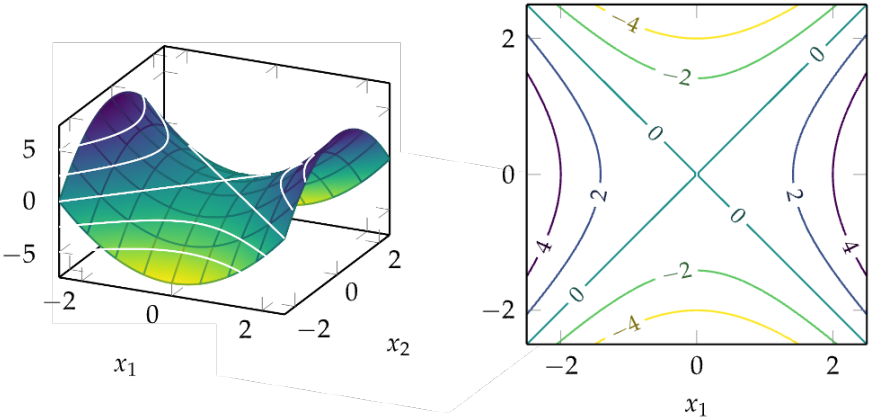
1. $\nabla f(\boldsymbol{x}^*) = 0$, the first-order necessary condition (FONC)
2. $\nabla^2 f(\boldsymbol{x}^*) \geq 0$, the second-order necessary condition (SONC)



Contour Plots

$$f(x_1, x_2) = x_1^2 - x_2^2$$

can be rendered in a 3D space but convenient to represent it also in 2D showing the lines of constant output value



Taylor Expansion

From the *first fundamental theorem of calculus*,[2] we know that

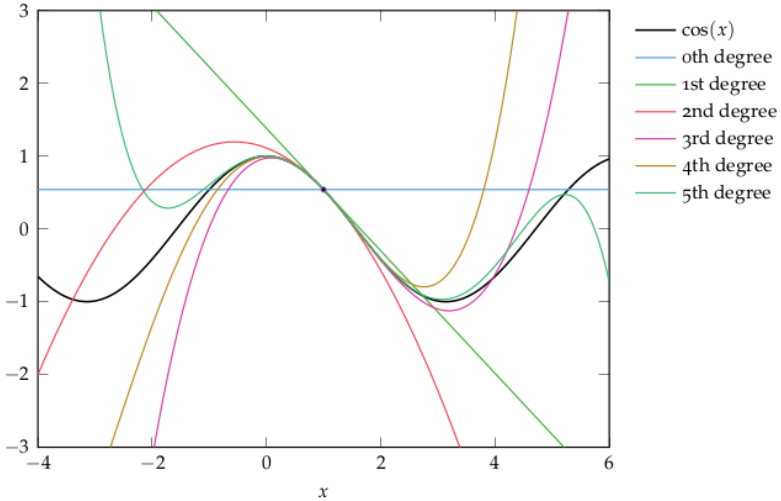
$$f(x+h) = f(x) + \int_0^h f'(x+a)\,da$$

Nesting this definition produces the Taylor expansion of f about x:

$$\begin{aligned}
f(x+h) &= f(x) + \int_0^h \left(f'(x) + \int_0^a f''(x+b)\,db \right) da \\
&= f(x) + f'(x)h + \int_0^h \int_0^a f''(x+b)\,db\,da \\
&= f(x) + f'(x)h + \int_0^h \int_0^a \left(f''(x) + \int_0^b f'''(x+c)\,dc \right) db\,da \\
&= f(x) + f'(x)h + \frac{f''(x)}{2!}h^2 + \int_0^h \int_0^a \int_0^b f'''(x+c)\,dc\,db\,da \\
&\vdots \\
&= f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \dots \\
&= \sum_{n=0}^{\infty} \frac{f^{(n)}(x)}{n!}h^n
\end{aligned}$$

Taylor Expansion

A smooth function looks like a polynomial when you zoom in closely enough.



Taylor Expansion for Multivariate Scalar-Valued Functions

The Taylor expansion of a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$ around a point $\boldsymbol{a}$ is given by:

In multiple dimensions, the Taylor expansion about $\mathbf{a}$ generalizes to

$$f(\mathbf{x}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top (\mathbf{x} - \mathbf{a}) + \frac{1}{2}(\mathbf{x} - \mathbf{a})^\top \nabla^2 f(\mathbf{a})(\mathbf{x} - \mathbf{a}) + \dots$$

- $\nabla f(\boldsymbol{a})$ is the gradient vector of f at $\boldsymbol{a}$, and
- $\nabla^2 f(\boldsymbol{a})$ is the Hessian matrix of f at $\boldsymbol{a}$.
- $O(||\boldsymbol{h}||^2)$ denotes the error by omitting higher-order terms. It becomes negligible as h approaches 0.

Example: Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

It has a global minimum at $(x, y) = (a, a^2)$, where $f(x, y) = 0$. Usually, these parameters are set such that $a = 1$ and $b = 100$. Only in the trivial case where $a = 0$ the function is symmetric and the minimum is at the origin.

Multivariate generalization
sum of $N/2$ uncoupled 2D Rosenbrock problems, and defined only for even N:

$$f(\mathrm{x}) = f(x_1, x_2, \ldots, x_N) = \sum_{i=1}^{N/2} \left[100(x_{2i-1}^2 - x_{2i})^2 + (x_{2i-1} - 1)^2\right].$$

This variant has predictably simple solutions.

Example: Rosenbrock function

Consider the Rosenbrock banana function,

$$f(\mathbf{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$$

Does the point $(1, 1)$ satisfy the FONC and SONC?

The gradient is:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2\left(10x_1^3 - 10x_1x_2 + x_1 - 1\right) \\ 10(x_2 - x_1^2) \end{bmatrix}$$

and the Hessian is:

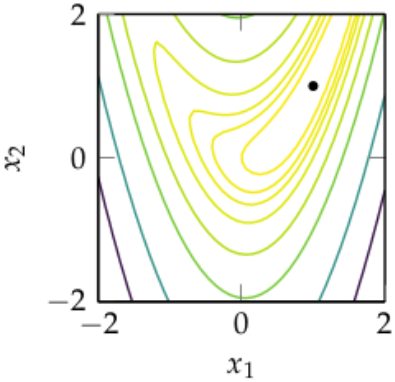
$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} \end{bmatrix} = \begin{bmatrix} -20(x_2 - x_1^2) + 40x_1^2 + 2 & -20x_1 \\ -20x_1 & 10 \end{bmatrix}$$

We compute $\nabla(f)([1, 1]) = 0$, so the FONC is satisfied. The Hessian at $[1, 1]$ is:

$$\begin{bmatrix} 42 & -20 \\ -20 & 10 \end{bmatrix}$$

which is positive definite, so the SONC is satisfied.

Example: Rosenbrock function



Overview

2. Derivatives → 4. Descent → 5. First-Order
3. Bracketing → 6. Second-Order

7. Direct

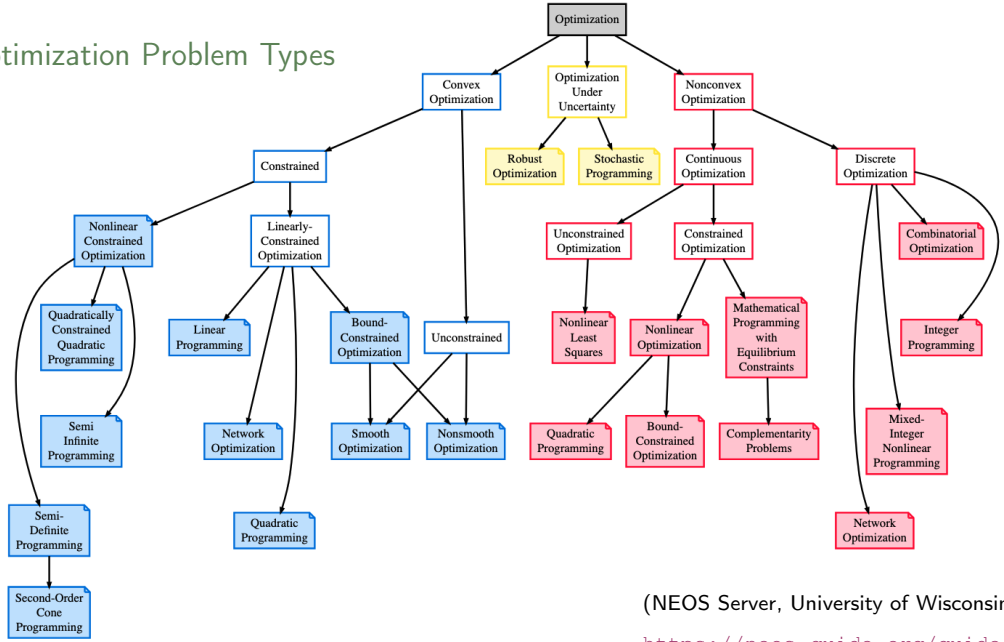
8. Stochastic → 9. Population → 20. Expressions
10. Constraints → 11. Linear → 19. Discrete
→ 12. Multiobjective

13. Sampling Plans → 14. Surrogate Models → 15. Probabilistic Surrogate Models → 16. Surrogate Optimization

17. Optimization under Uncertainty → 18. Uncertainty Propagation

21. Multidisciplinary Design Optimization

Optimization Problem Types



(NEOS Server, University of Wisconsin)

https://neos-guide.org/guide/types/

Problem classification

Different classification factors:

- Univariate $f : \mathbb{R} \to \mathbb{R}$ vs Multivariate $f : \mathbb{R}^n \to \mathbb{R}$
- Scalar-valued $f : \mathbb{R}^n \to \mathbb{R}$ vs vector-vector functions $f : \mathbb{R}^n \to \mathbb{R}^m$
- Linear vs Nonlinear
- Nonlinear: Convex vs Nonconvex, unimodal vs multimodal
- Constrained vs unconstrained
- Smooth (differentiable) vs non smooth (non differentiable)
- Continuous vs Discrete
- Deterministic vs Uncertain

Application Example

In robotics, an autonomous agent (e.g., a drone, robotic arm, or self-driving car) needs to determine an **optimal path** from a starting position to a target while satisfying constraints such as avoiding obstacles, minimizing energy consumption, or optimizing smoothness.

Formulating the Optimization Problem

The goal is to find a smooth trajectory $x(t)$ that minimizes a cost function:

$$J = \int_{t_0}^{t^f} C(\boldsymbol{x}(t), \boldsymbol{u}(t))dt$$

$\boldsymbol{x}(t)$ is the state (e.g., position, velocity), $\boldsymbol{u}(t)$ is the control input (e.g., force, acceleration), $C(\boldsymbol{x}, \boldsymbol{u})$ is the cost function, which could represent energy usage, time, or distance.

Constraints:

- Dynamic Constraints: Governed by the system's physics (e.g., Newton's laws for a robot arm or quadcopter): $\boldsymbol{x}' = f(\boldsymbol{x}, \boldsymbol{u})$
- Obstacle Avoidance: Ensures that the trajectory does not collide with obstacles: $g(\boldsymbol{x}(t)) \geq 0, \forall t$
- Boundary Conditions: The system must start and end at given positions with certain velocities.

Summary

- Optimization is the process of finding the best system design subject to a set of constraints
- Optimization is concerned with finding global minima of a function
- Minima can occur where the gradient is zero, but zero-gradient does not imply optimality

2. Derivatives and Gradients

Definitions

- $[a, b] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$ closed interval
  $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$ open interval
- We denote vectors in bold, eg, $\boldsymbol{x}$ and matrices in uppercase letters, eg, A
- Given a vector $\boldsymbol{x} \in \mathbb{R}^n$, x_i is its i-th component
- We invariably assume that $\boldsymbol{x}$ is a **column vector**,
  which we also write with parentheses as $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)$ or $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]$.
- The transpose $\boldsymbol{x}^T$ is a row vector

$$\boldsymbol{x}^T = \begin{bmatrix} x_1 & x_2 & \ldots & x_n \end{bmatrix}$$

- Scalar product: $\boldsymbol{y}^T \boldsymbol{x} = \sum_{i=1}^{n} y_i x_i$

Definitions

- **linear combination**

$$\begin{aligned} \boldsymbol{v}_1, \boldsymbol{v}_2 \ldots, \boldsymbol{v}_k &\in \mathbb{R}^n \\ \boldsymbol{\lambda} = [\lambda_1, \ldots, \lambda_k]^T &\in \mathbb{R}^k \end{aligned} \qquad \boldsymbol{x} = \lambda_1 \boldsymbol{v}_1 + \cdots + \lambda_k \boldsymbol{v}_k = \sum_{i=1}^{k} \lambda_i \boldsymbol{v}_i$$

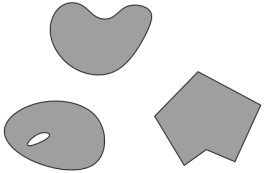
moreover:

$\boldsymbol{\lambda} \geq 0$	**conic combination**	
$\boldsymbol{\lambda}^T 1 = 1$	**affine combination**	$\left(\sum_{i=1}^{k} \lambda_i = 1 \right)$
$\boldsymbol{\lambda} \geq 0$ and $\boldsymbol{\lambda}^T 1 = 1$	**convex combination**	

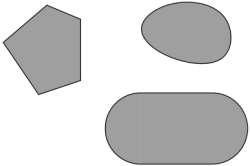
- $A\boldsymbol{x}$ column vector combination of the columns of A;
  $\boldsymbol{u}^T A$ row vector combination of the rows of A

Definitions

- **convex set**: if $\boldsymbol{x}, \boldsymbol{y} \in S$ and $0 \leq \lambda \leq 1$ then $\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y} \in S$

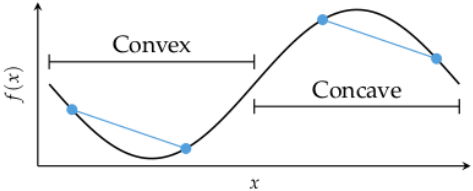


nonconvex



convex

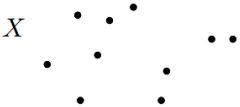
- **convex function**
  - if its epigraph $\{(x, y) \in \mathbb{R}^2 : y \geq f(x)\}$ is a convex set or
  - if $f : \mathbb{R}^n \to \mathbb{R}$ and if $\forall \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n, \alpha \in [0, 1]$ it holds that $f(\alpha \boldsymbol{x} + (1-\alpha)\boldsymbol{y}) \leq \alpha f(\boldsymbol{x}) + (1-\alpha) f(\boldsymbol{y})$

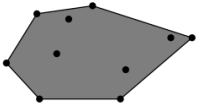


Definitions

- For a set of points $S \subseteq \mathbb{R}^n$

$\mathrm{lin}(S)$	linear hull (span)
$\mathrm{cone}(S)$	conic hull
$\mathrm{aff}(S)$	affine hull
$\mathrm{conv}(S)$	convex hull





the convex hull of X

$$\mathrm{conv}(X) = \left\{ \lambda_1 x_1 + \lambda_2 x_2 + \ldots + \lambda_n x_n \;\middle|\; x_i \in X,\ \lambda_1, \ldots, \lambda_n \geq 0 \text{ and } \sum_i \lambda_i = 1 \right\}$$

Norms

<u>Def.</u> A **norm** is a function that assigns a length to a vector.

A function f is a norm if:

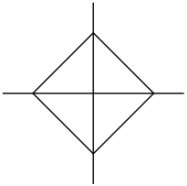
1. $f(\boldsymbol{x}) = 0$ if and only if $\boldsymbol{x}$ is the zero vector
2. $f(a\boldsymbol{x}) = |a| f(\boldsymbol{x})$, such that lengths scale
3. $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$, also known as triangle inequality

L_p norms are commonly used set of norms parameterized by a scalar $p \geq 1$:

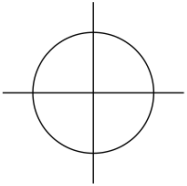
$$\|\boldsymbol{x}\|_p = \lim_{\rho \to p} \left(|x_1|^\rho + |x_2|^\rho + \ldots + |x_n|^\rho\right)^{\frac{1}{\rho}}$$

L_∞ is also called the max norm, Chebyshev distance or chessboard distance.

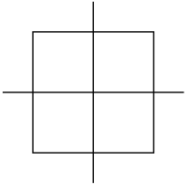
L_1: $\|\mathbf{x}\|_1 = |x_1| + |x_2| + \cdots + |x_n|$



L_2: $\|\mathbf{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}$



L_∞: $\|\mathbf{x}\|_\infty = \max(|x_1|, |x_2|, \cdots, |x_n|)$



Matrix Norms

$$\|A\| \stackrel{\text{def}}{=} \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}. \tag{A.7}$$

The matrix norms defined in this way are said to be *consistent* with the vector norms (A.2). Explicit formulae for these norms are as follows:

$$\|A\|_1 = \max_{j=1,\ldots,n} \sum_{i=1}^{m} |A_{ij}|, \tag{A.8a}$$

$$\|A\|_2 = \text{largest eigenvalue of } (A^T A)^{1/2}, \tag{A.8b}$$

$$\|A\|_\infty = \max_{i=1,\ldots,m} \sum_{j=1}^{n} |A_{ij}|. \tag{A.8c}$$

The Frobenius norm $\|A\|_F$ of the matrix A is defined by

$$\|A\|_F = \left(\sum_{i=1}^{m} \sum_{j=1}^{n} A_{ij}^2 \right)^{1/2}. \tag{A.9}$$

Condition Number

For a nonsingular matrix A, the **condition number** $\kappa(A)$ is defined as

$$\kappa(A) = \|A\| \left\|A^{-1}\right\|$$

where $\|\cdot\|$ is any matrix norm. The condition number of a matrix is a measure of how close the matrix is to being singular. A matrix with a large condition number is said to be ill-conditioned, and small changes in the input can lead to large changes in the output when solving linear systems or performing matrix operations involving that matrix.

Outline

3. Derivatives

4. Symbolic Differentiation

5. Numerical Differentiation

6. Automatic Differentiation

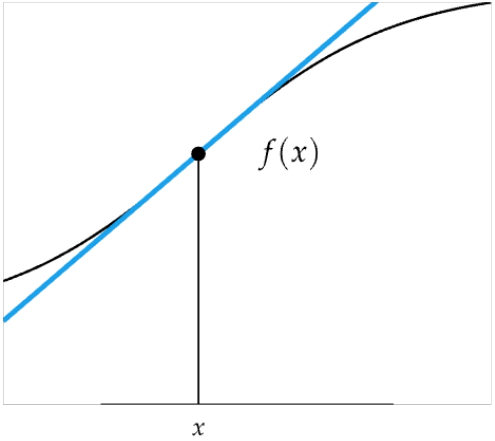
Derivatives

- Derivatives tell us which direction to search for a solution

- Slope of Tangent Line

$$f'(x) := \frac{\mathrm{d}f(x)}{\mathrm{d}x}$$

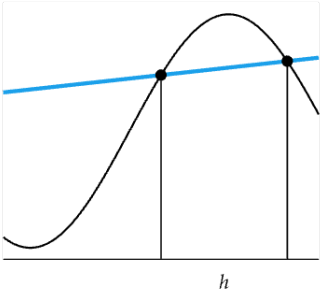
(Leibniz notation)

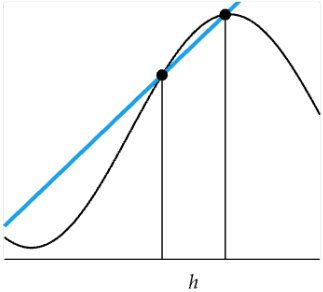


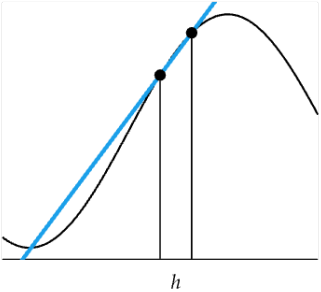
Derivatives

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x$$

$$f'(x) = \frac{\Delta f(x)}{\Delta x}$$







Symbolic Differentiation

$$f'(x) \equiv \underbrace{\lim_{h\to 0} \frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} = \underbrace{\lim_{h\to 0} \frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h\to 0} \frac{f(x)-f(x-h)}{h}}_{\text{backward difference}}$$

Symbolic Differentiation

```python
import sympy as sp

# Define the variable
x = sp.symbols('x')

# Define the function
f = x**2 + x/2 - sp.sin(x)/x

# Compute the derivative
df_dx = sp.diff(f, x)

# Display the result
print("The symbolic derivative of f is:")
print(df_dx)
```

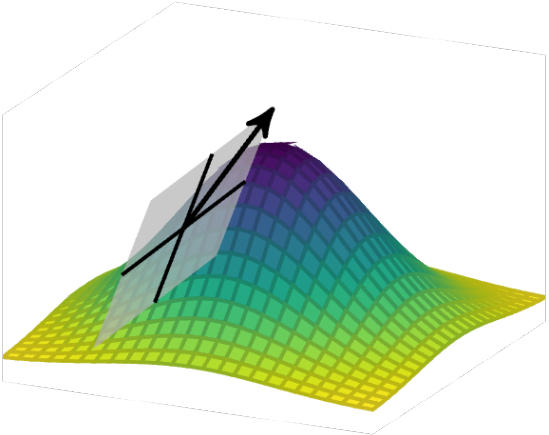
Derivatives in Multiple Dimensions

- Gradient Vector

$$\nabla f(\boldsymbol{x})^T = \begin{bmatrix} \frac{\partial f(\boldsymbol{x})}{\partial x_1} & \frac{\partial f(\boldsymbol{x})}{\partial x_2} & \cdots & \frac{\partial f(\boldsymbol{x})}{\partial x_n} \end{bmatrix}$$

- Hessian Matrix

$$\nabla^2 f(\boldsymbol{x}) = \begin{bmatrix} \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_1} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_2} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2 \partial x_2} & \cdots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2 \partial x_n} \\ \vdots & \ddots & & \vdots \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_n} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2 \partial x_n} & \cdots & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_n \partial x_n} \end{bmatrix}$$



Directional derivative

The **directional derivative** $\nabla_{\boldsymbol{s}} f(\boldsymbol{x})$ of a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$ is the instantaneous rate of change of $f(\boldsymbol{x})$ as $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]$ is moved with velocity $\boldsymbol{s} = [s_1, s_2, \ldots, s_n]$.

$$\nabla_{\mathbf{s}} f(\mathbf{x}) \equiv \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}) - f(\mathbf{x})}{h}}_{\text{forward difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x} + h\mathbf{s}/2) - f(\mathbf{x} - h\mathbf{s}/2)}{h}}_{\text{central difference}} = \underbrace{\lim_{h \to 0} \frac{f(\mathbf{x}) - f(\mathbf{x} - h\mathbf{s})}{h}}_{\text{backward difference}}$$

To compute $\nabla_{\boldsymbol{s}} f(\boldsymbol{x})$:

- compute $\boxed{\nabla_{\boldsymbol{s}} f(\boldsymbol{x}) = \frac{\partial f(\boldsymbol{x})}{\partial x_1} s_1 + \frac{\partial f(\boldsymbol{x})}{\partial x_2} s_2 + \ldots + \frac{\partial f(\boldsymbol{x})}{\partial x_n} s_n = \nabla f(\boldsymbol{x})^T \boldsymbol{s} = \nabla f(\boldsymbol{x}) \cdot \boldsymbol{s}}$
- $g(\alpha) := f(\boldsymbol{x} + \alpha \boldsymbol{s})$ and then compute $g'(0)$

We wish to compute the directional derivative of $f(\mathbf{x}) = x_1 x_2$ at $\mathbf{x} = [1, 0]$ in the direction $\mathbf{s} = [-1, -1]$:

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \quad \frac{\partial f}{\partial x_2}\right] = [x_2, x_1]$$

$$\nabla_{\mathbf{s}} f(\mathbf{x}) = \nabla f(\mathbf{x})^\top \mathbf{s} = \begin{bmatrix} 0 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -1$$

We can also compute the directional derivative as follows:

$$g(\alpha) = f(\mathbf{x} + \alpha \mathbf{s}) = (1 - \alpha)(-\alpha) = \alpha^2 - \alpha$$

$$g'(\alpha) = 2\alpha - 1$$

$$g'(0) = -1$$

Matrix Calculus

Common gradient:

$$\nabla_{\boldsymbol{x}} \boldsymbol{b}^T \boldsymbol{x} = ?$$

$$\boldsymbol{b}^T \boldsymbol{x} = [b_1 x_1 + b_2 x_2 + \ldots + b_n x_n]$$

$$\frac{\partial \boldsymbol{b}^T \boldsymbol{x}}{\partial x_i} = b_i$$

$$\nabla_{\boldsymbol{x}} \boldsymbol{b}^T \boldsymbol{x} = \nabla_{\boldsymbol{x}} \boldsymbol{x}^T \boldsymbol{b} = \boldsymbol{b}$$

Matrix Calculus

Common gradient:

$$\nabla_{\boldsymbol{x}} \boldsymbol{x}^T A \boldsymbol{x} = ?$$

$$\boldsymbol{x}^T A \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}^T \begin{bmatrix} x_1 a_{11} + x_2 a_{12} + \dots + x_n a_{1n} \\ x_1 a_{21} + x_2 a_{22} + \dots + x_n a_{2n} \\ \vdots \\ x_1 a_{n1} + x_2 a_{n2} + \dots + x_n a_{nn} \end{bmatrix}$$

$$= \begin{matrix} x_1^2 a_{11} + x_1 x_2 a_{12} + \dots + x_1 x_n a_{1n} + \\ x_1 x_2 a_{21} + x_2^2 a_{22} + \dots + x_2 x_n a_{2n} + \\ \vdots \\ x_1 x_n a_{n1} + x_2 x_n a_{n2} + \dots + x_n^2 a_{nn} \end{matrix}$$

$$\frac{\partial}{\partial x_i} \boldsymbol{x}^T A \boldsymbol{x} = \sum_{j=1}^{n} x_j \left(a_{ij} + a_{ji}\right)$$

$$\nabla_x \boldsymbol{x}^T A \boldsymbol{x} = \begin{bmatrix} \sum_{j=1}^{n} x_j \left(a_{1j} + a_{j1}\right) \\ \sum_{j=1}^{n} x_j \left(a_{2j} + a_{j2}\right) \\ \vdots \\ \sum_{j=1}^{n} x_j \left(a_{nj} + a_{jn}\right) \end{bmatrix} = \begin{bmatrix} a_{11} + a_{11} & a_{12} + a_{21} & \dots & a_{1n} + a_{n1} \\ a_{21} + a_{12} & a_{22} + a_{22} & \dots & a_{2n} + a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} + a_{1n} & a_{n2} + a_{2n} & \dots & a_{nn} + a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \left(A + A^T\right) \boldsymbol{x}$$

Smoothness

<u>Def.</u> The **smoothness** of a function is a property measured by the number of continuous derivatives (differentiability class) it has over its domain.

A function of **class** C^k is a function of smoothness at least k; that is, a function of class C^k is a function that has a kth derivative that is continuous in its domain.

The term **smooth function** refers to a C^∞-function. However, it may also mean "sufficiently differentiable" for the problem under consideration.

Smoothness

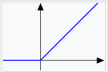
- Let U be an open set on the real line and a function f defined on U with real values. Let k be a non-negative integer.
- The function f is said to be of **differentiability class** C^k if the derivatives $f', f'', \ldots, f^{(k)}$ exist and are continuous on U.
- If f is k-differentiable on U, then it is at least in the class C^{k-1} since $f', f'', \ldots, f^{(k-1)}$ are continuous on U.
- The function f is said to be **infinitely differentiable**, **smooth**, or of **class** C^∞, if it has derivatives of all orders (continuous) on U.
- The function f is said to be of **class** C^ω, or **analytic**, if f is smooth and its Taylor series expansion around any point in its domain converges to the function in some neighborhood of the point.
- There exist functions that are smooth but not analytic; C^ω is thus strictly contained in C^∞. Bump functions are examples of functions with this property.

Example: continuous (C^0) but not differentiable [edit]

The function

$$f(x) = \begin{cases} x & \text{if } x \geq 0, \\ 0 & \text{if } x < 0 \end{cases}$$

is continuous, but not differentiable at $x = 0$, so it is of class C^0, but not of class C^1.



The C^0 function $f(x) = x$ for $x \geq 0$ and 0 otherwise.

Example: finitely-times differentiable (C^k) [edit]

For each even integer k, the function

$$f(x) = |x|^{k+1}$$

is continuous and k times differentiable at all x. At $x = 0$, however, f is not $(k + 1)$ times differentiable, so f is of class C^k, but not of class C^j where $j > k$.

The function $g(x) = x^2 \sin(1/x)$ for $x > 0$.

Example: differentiable but not continuously differentiable (not C^1) [edit]

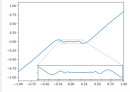
The function

$$g(x) = \begin{cases} x^2 \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0 \end{cases}$$

is differentiable, with derivative

$$g'(x) = \begin{cases} -\cos\left(\frac{1}{x}\right) + 2x \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0. \end{cases}$$

Because $\cos(1/x)$ oscillates as $x \to 0$, $g'(x)$ is not continuous at zero. Therefore, $g(x)$ is differentiable but not of class C^1.



The function $f : \mathbb{R} \to \mathbb{R}$ with $f(x) = x^2 \sin\left(\frac{1}{x}\right)$ for $x \neq 0$ and $f(0) = 0$ is differentiable. However, this function is not continuously differentiable.

Example: differentiable but not Lipschitz continuous [edit]

The function

$$h(x) = \begin{cases} x^{4/3} \sin\left(\frac{1}{x}\right) & \text{if } x \neq 0, \\ 0 & \text{if } x = 0 \end{cases}$$

is differentiable but its derivative is unbounded on a compact set. Therefore, h is an example of a function that is differentiable but not locally Lipschitz continuous.

Example: analytic (C^ω) [edit]

The exponential function e^x is analytic, and hence falls into the class C^ω (where ω is the smallest transfinite ordinal). The trigonometric functions are also analytic wherever they are defined, because they are linear combinations of complex exponential functions e^{ix} and e^{-ix}.

A smooth function that is not analytic.

Example: smooth (C^∞) but not analytic (C^ω) [edit]

The bump function

$$f(x) = \begin{cases} e^{-\frac{1}{1-x^2}} & \text{if } |x| < 1, \\ 0 & \text{otherwise} \end{cases}$$

is smooth, so of class C^∞, but it is not analytic at $x = \pm 1$, and hence is not of class C^ω. The function f is an example of a smooth function with compact support.

Positive Definteness

<u>Def.</u> A square, symmetric matrix A is **positive definite** if $\boldsymbol{x}^T A\boldsymbol{x}$ is positive for all points other than the origin: $\boldsymbol{x}^T A\boldsymbol{x} > 0$ for all $x \neq 0$.
<u>Def.</u> A square, symmetric matrix A is **positive semidefinite** if $\boldsymbol{x}^T A\boldsymbol{x}$ is always non-negative: $\boldsymbol{x}^T A\boldsymbol{x} \geq 0$ for all $\boldsymbol{x}$.

A matrix A is positive definite if and only if all its eigenvalues are positive. (This can be used for recognition.)

If the matrix A is positive definite in the function $f(\boldsymbol{x}) = \boldsymbol{x}^T A\boldsymbol{x}$, then f has a unique global minimum.

Recall that the second order Taylor approximation of a twice-differentiable function f at $\boldsymbol{x}_0$ is

$$f(\boldsymbol{x}) \approx f(\boldsymbol{x}_0) + \nabla f(\boldsymbol{x}_0)^T(\boldsymbol{x} - \boldsymbol{x}_0) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_0)^T H_0(\boldsymbol{x} - \boldsymbol{x}_0)$$

where H_0 is the Hessian evaluated at $\boldsymbol{x}_0$. If $(\boldsymbol{x} - \boldsymbol{x}_0)^T H_0(\boldsymbol{x} - \boldsymbol{x}_0)$ has a unique global minimum, then the overall approximation has a unique global minimum.

Non-symmetric matrices can be positive definite:
Every real matrix A decomposes uniquely as a symmetric matrix and a skew-symmetric matrix:

$$A = S + K, \quad S = \frac{1}{2}(A + A^T), \quad K = \frac{1}{2}(A - A^T).$$

Then:

$$\boldsymbol{x}^T A x = \boldsymbol{x}^T S \boldsymbol{x},$$

because

$$\boldsymbol{x}^T K \boldsymbol{x} = 0 \text{ for all } \boldsymbol{x}.$$

However, all local optimization algorithms are based, explicitly or implicitly, on the second-order Taylor expansion. The matrix governing local curvature is the Hessian and Hessians are necessarily symmetric

LU Decomposition

The LU factorization of a matrix $A \in \mathbf{R}^{n \times n}$ is defined as

$$PA = LU, \tag{A.20}$$

where

P is an $n \times n$ permutation matrix (that is, it is obtained by rearranging the rows of the $n \times n$ identity matrix),

L is unit lower triangular (that is, lower triangular with diagonal elements equal to 1, and

U is upper triangular.

- This factorization can be used to solve efficiently a linear system of the form $A\boldsymbol{x} = \boldsymbol{b}$ by first solving $L\boldsymbol{y} = \boldsymbol{b}$ for y and then solving $U\boldsymbol{x} = \boldsymbol{y}$ for $\boldsymbol{x}$.
- It can be found by using Gaussian elimination with row partial pivoting, an algorithm that requires approximately $2n^3/3$ floating-point operations when A is dense. Done by standard software LAPACK.

Cholesky Decomposition

When $A \in \mathbf{R}^{n \times n}$ is symmetric positive definite, it is possible to compute a similar but more specialized factorization at about half the cost—about $n^3/3$ operations. This factorization, known as the Cholesky factorization, produces a matrix L such that

$$A = LL^T. \tag{A.23}$$

(If we require L to have positive diagonal elements, it is uniquely defined by this formula.)

- It can also be used to solve systems of linear equations
- The Cholesky factorization can also be used to verify positive definiteness of a symmetric matrix A.

Outline

3. Derivatives

4. Symbolic Differentiation

5. Numerical Differentiation

6. Automatic Differentiation

Symbolic Derivatives

- Symbolic derivatives can give valuable insight into the structure of the problem domain and, in some cases, produce analytical solutions of extrema (e.g., solving for $\frac{\mathrm{d}}{\mathrm{d}x} f(x) = 0$) that can eliminate the need for derivative calculation altogether.

- But they do not lend themselves to efficient runtime calculation of derivative values, as they can get exponentially larger than the expression whose derivative they represent

Outline

3. Derivatives

4. Symbolic Differentiation

5. Numerical Differentiation

6. Automatic Differentiation

Numerical Differentiation

Finite Difference Method

- Neighboring points are used to approximate the derivative

$$f'(x) \approx \underbrace{\frac{f(x+h)-f(x)}{h}}_{\text{forward difference}} \approx \underbrace{\frac{f(x+h/2)-f(x-h/2)}{h}}_{\text{central difference}} \approx \underbrace{\frac{f(x)-f(x-h)}{h}}_{\text{backward difference}}$$

- h too small causes numerical cancellation errors (square root or cube root of the machine precision for floating point values: `sys.float_info.epsilon` difference between 1 and closest representable number)

Derivation

from Taylor series expansion:

$$f(x+h) = f(x) + \frac{f'(x)}{1!}h + \frac{f''(x)}{2!}h^2 + \frac{f'''(x)}{3!}h^3 + \cdots$$

We can rearrange and solve for the first derivative:

$$f'(x)h = f(x+h) - f(x) - \frac{f''(x)}{2!}h^2 - \frac{f'''(x)}{3!}h^3 - \cdots$$

$$f'(x) = \frac{f(x+h) - f(x)}{h} - \frac{f''(x)}{2!}h - \frac{f'''(x)}{3!}h^2 - \cdots$$

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

- forward difference has error term $O(h)$, linear error as h approaches zero
- central difference has error term is $O(h^2)$

```python
import sys
import numpy as np

def diff_forward(f, x: float, h: float=np.sqrt(sys.float_info.epsilon)) -> float:
    return (f(x+h) - f(x))/h

def diff_central(f, x: float, h: float=np.cbrt(sys.float_info.epsilon)) -> float:
    return (f(x+h/2) - f(x-h/2))/h

def diff_backward(f, x: float, h: float=np.sqrt(sys.float_info.epsilon)) -> float:
    return (f(x) - f(x-h))/h

# Example usage
def func(x):
    return x**2 + np.sin(x)

x0 = 1.0
print(f"The derivative at x = {x0} is {diff_forward(func, x0)}")
```

Numerical Differentiation

Complex step method

Uses one single function evaluation after taking a step in the imaginary direction.

$$f(x+ih) = f(x) + ihf'(x) - h^2\frac{f''(x)}{2!} - ih^3\frac{f'''(x)}{3!} + \cdots$$

$$\begin{aligned}
\mathrm{Im}(f(x+ih)) &= hf'(x) - h^3\frac{f'''(x)}{3!} + \cdots \\
\Rightarrow f'(x) &= \frac{\mathrm{Im}(f(x+ih))}{h} + h^2\frac{f'''(x)}{3!} - \cdots \\
&= \frac{\mathrm{Im}(f(x+ih))}{h} + O(h^2) \text{ as } h \to 0
\end{aligned}$$

$$\mathrm{Re}(f(x+ih)) = f(x) - h^2\frac{f''(x)}{2!} + \ldots$$

$$\Rightarrow f(x) = \mathrm{Re}(f(x+ih)) + h^2\frac{f''(x)}{2!} - \cdots$$

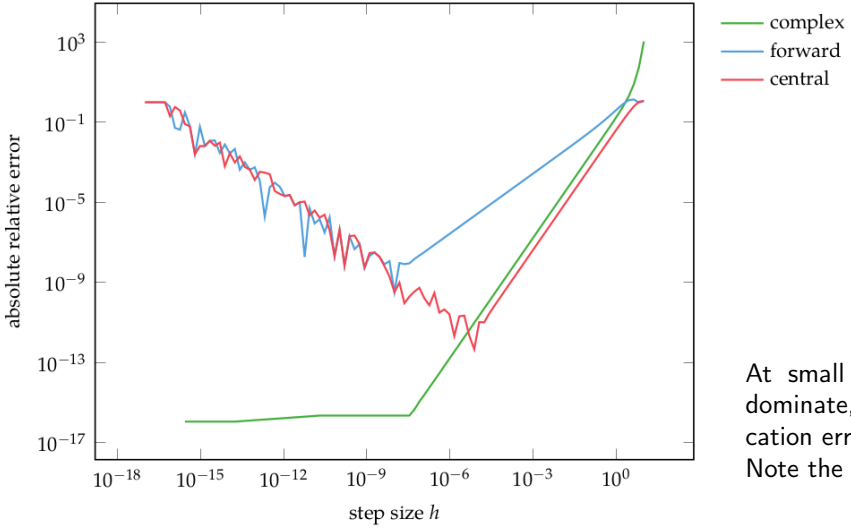
```python
import numpy as np

def diff_complex(f, x: float, h: float=1e-20) -> float:
    return np.imag(f(x + h * 1j)) / h

# Example usage
def func(x):
    return x**2 + np.sin(x)

x0 = 1.0
print(f"The derivative at x = {x0} is {diff_complex(func, x0)}")
```

Numerical Differentiation Error Comparison



At small h, round off errors dominate, and at large h, truncation errors dominate.
Note the log transformation.

Numerical Differentiation in ML

- Approximation errors would be tolerated in a deep learning setting thanks to the well-documented error resiliency of neural network architectures (Gupta et al., 2015).
- The $O(n)$ complexity of numerical differentiation for a gradient in n dimensions is the main obstacle to its usefulness in machine learning, where n can be as large as millions or billions in state-of-the-art deep learning models (Shazeer et al., 2017).

Outline

3. Derivatives

4. Symbolic Differentiation

5. Numerical Differentiation

6. Automatic Differentiation

Automatic Differentiation

Automatic differentiation techniques are founded on the observation that any function is evaluated by performing a sequence of simple elementary operations involving just one or two arguments at a time:

- addition
- multiplication
- division
- power operation a^b
- trigonometric functions
- exponential functions
- logarithmic
- chain rule:

Chain Rule

The chain rule is a formula that expresses the derivative of the composition of two differentiable functions. More precisely, if $h = z \circ y$ is the composition such that $h(x) = z(y(x))$ for every x, then the chain rule is:

in Lagrange's notation:

$$h'(x) = z'(y(x))y'(x).$$

in Leibniz's notation:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx},$$

and

$$\left.\frac{dz}{dx}\right|_x = \left.\frac{dz}{dy}\right|_{y(x)} \cdot \left.\frac{dy}{dx}\right|_x,$$

for indicating at which points the derivatives have to be evaluated.

- Forward Accumulation is equivalent to expanding a function using the chain rule and computing the derivatives inside-out
- Requires n-passes to compute n-dimensional gradient
- Example:

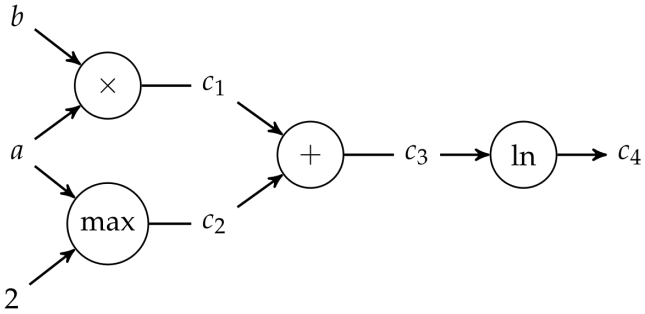
$$f(a, b) = \ln(ab + \max(a, 2))$$

$$\begin{aligned}
\frac{\partial f}{\partial a} &= \frac{\partial}{\partial a} \ln(ab + \max(a, 2)) \\
&= \frac{1}{ab + \max(a, 2)} \frac{\partial}{\partial a}(ab + \max(a, 2)) \\
&= \frac{1}{ab + \max(a, 2)} \left[\frac{\partial(ab)}{\partial a} + \frac{\partial \max(a, 2)}{\partial a} \right] \\
&= \frac{1}{ab + \max(a, 2)} \left[\left(b\frac{\partial a}{\partial a} + a\frac{\partial b}{\partial a} \right) + \left((2 > a)\frac{\partial 2}{\partial a} + (2 < a)\frac{\partial a}{\partial a} \right) \right] \\
&= \frac{1}{ab + \max(a, 2)} [b + (2 < a)]
\end{aligned}$$

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a, b) = \ln(ab + \max(a, 2))$



Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

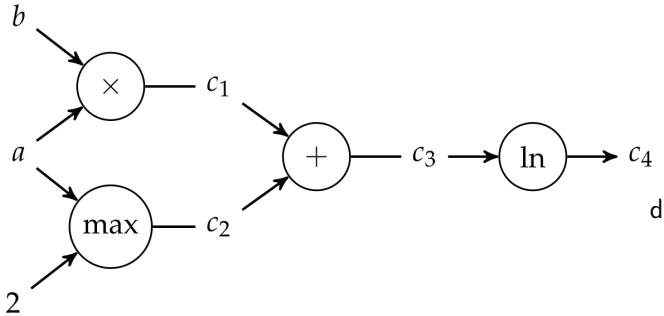
Forward accumulation for $f(a, b) = \ln(ab + \max(a, 2))$

$b = 2$

$\dot{b} = 0$

$a = 3$

$\dot{a} = 1$



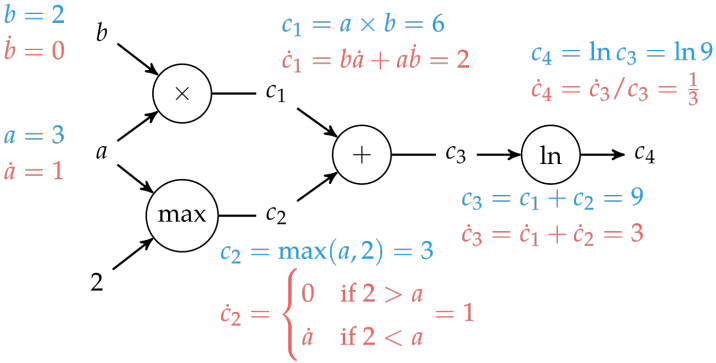
dual numbers

$\frac{\partial b}{\partial a} = \dot{b}$ Newton notation

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a, b) = \ln(ab + \max(a, 2))$



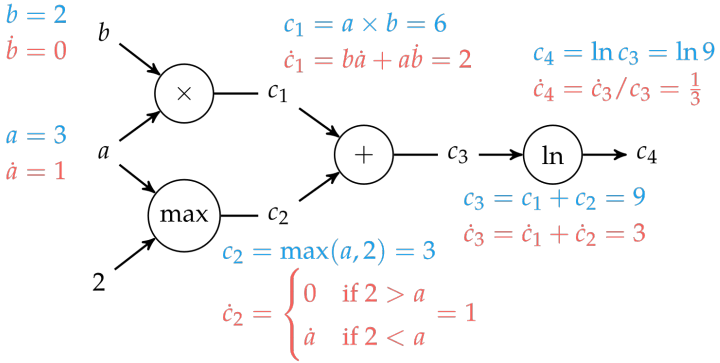
dual numbers

$\frac{\partial b}{\partial a} = \dot{b}$ Newton notation

Automatic Differentiation

Computational graph: nodes are are operations and the edges are input-output relations. leaf nodes of a computational graph are input variables or constants, and terminal nodes are values output by the function

Forward accumulation for $f(a, b) = \ln(ab + \max(a, 2))$



dual numbers

$\frac{\partial b}{\partial a} = \dot{b}$ Newton notation

for $\frac{\partial f}{\partial b}$ set $\dot{a} = 0, \dot{b} = 1$

Dual numbers

- Dual numbers can be expressed mathematically by including the abstract quantity ϵ, where ϵ^2 is defined to be 0.

- Like a complex number, a dual number is written $a + b\epsilon$ where a and b are both real values.

- $(a + b\epsilon) + (c + d\epsilon) = (a + c) + (b + d)\epsilon$
  $(a + b\epsilon) \times (c + d\epsilon) = (ac) + (ad + bc)\epsilon$

- by passing a dual number into any smooth function f, we get the evaluation and its derivative. We can show this using the Taylor series:

$$
\begin{aligned}
f(x) &= \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(x-a)^k \\
f(a+b\epsilon) &= \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!}(a+b\epsilon-a)^k \\
&= \sum_{k=0}^{\infty} \frac{f^{(k)}(a)b^k\epsilon^k}{k!} \\
&= f(a) + bf'(a)\epsilon + \epsilon^2 \sum_{k=2}^{\infty} \frac{f^{(k)}(a)b^k}{k!}\epsilon^{(k-2)} \\
&= f(a) + bf'(a)\epsilon
\end{aligned}
$$

Note that

$$
\begin{aligned}
(v + \dot{v}\epsilon) + (u + \dot{u}\epsilon) &= (v + u) + (\dot{v} + \dot{u})\epsilon \\
(v + \dot{v}\epsilon)(u + \dot{u}\epsilon) &= (vu) + (v\dot{u} + \dot{v}u)\epsilon \ ,
\end{aligned}
$$

satisfies the rules of differentiation

Setting:

$$
f(v + \dot{v}\epsilon) = f(v) + f'(v)\dot{v}\epsilon
$$

The chain rule follows:

$$
\begin{aligned}
f(g(v + \dot{v}\epsilon)) &= f(g(v) + g'(v)\dot{v}\epsilon) \\
&= f(g(v)) + f'(g(v))g'(v)\dot{v}\epsilon \ .
\end{aligned}
$$

Automatic Differentiation

- **Reverse accumulation** is performed in a single run using two passes $O(m \cdot \text{ops}(f))$ (forward and back) for $f : \mathbb{R}^n \to \mathbb{R}^m$
- Note: this is central to the back-propagation algorithm used to train neural networks because it needs only one pass for the n-dimensional function to find the gradient.
- implemented through two different operation overloading functions (for forward and backward)
- Many open-source software implementations are available: eg, Tensorflow

Forward implements:

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\frac{dc_3}{dx}\right) = \frac{df}{dc_4}\left(\frac{dc_4}{dc_3}\left(\frac{dc_3}{dc_2}\frac{dc_2}{dx} + \frac{dc_3}{dc_1}\frac{dc_1}{dx}\right)\right)$$

Backward implements:

$$\frac{df}{dx} = \frac{df}{dc_4}\frac{dc_4}{dx} = \left(\frac{df}{dc_3}\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx} = \left(\left(\frac{df}{dc_2}\frac{dc_2}{dc_3} + \frac{df}{dc_1}\frac{dc_1}{dc_3}\right)\frac{dc_3}{dc_4}\right)\frac{dc_4}{dx}$$

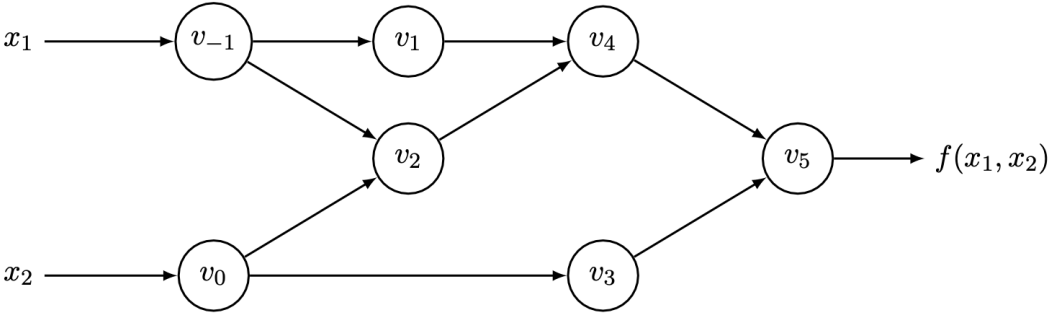
Complementing each intermediate variable v_i with an **adjoint**

$$\bar{v}_i = \frac{\partial y_j}{\partial v_i}$$

which represents the sensitivity of a considered output y_j with respect to changes in v_i.

Example

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - sin(x_2)$$



Example: Forward Accumulation

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - sin(x_2)$$

Forward Primal Trace		
v_{-1}	$= x_1$	$= 2$
v_0	$= x_2$	$= 5$
v_1	$= \ln v_{-1}$	$= \ln 2$
v_2	$= v_{-1} \times v_0$	$= 2 \times 5$
v_3	$= \sin v_0$	$= \sin 5$
v_4	$= v_1 + v_2$	$= 0.693 + 10$
v_5	$= v_4 - v_3$	$= 10.693 + 0.959$
y	$= v_5$	$= 11.652$

Forward Tangent (Derivative) Trace		
$\dot{v}_{-1}$	$= \dot{x}_1$	$= 1$
$\dot{v}_0$	$= \dot{x}_2$	$= 0$
$\dot{v}_1$	$= \dot{v}_{-1}/v_{-1}$	$= 1/2$
$\dot{v}_2$	$= \dot{v}_{-1} \times v_0 + \dot{v}_0 \times v_{-1}$	$= 1 \times 5 + 0 \times 2$
$\dot{v}_3$	$= \dot{v}_0 \times \cos v_0$	$= 0 \times \cos 5$
$\dot{v}_4$	$= \dot{v}_1 + \dot{v}_2$	$= 0.5 + 5$
$\dot{v}_5$	$= \dot{v}_4 - \dot{v}_3$	$= 5.5 - 0$
$\dot{y}$	$= \dot{v}_5$	$= 5.5$

$$O(n \cdot \mathrm{ops}(f))$$

Example: Reverse Accumulation

Forward Primal Trace

$$
\begin{aligned}
v_{-1} &= x_1 && = 2 \\
v_0 &= x_2 && = 5 \\
\hline
v_1 &= \ln v_{-1} && = \ln 2 \\
v_2 &= v_{-1} \times v_0 && = 2 \times 5 \\
v_3 &= \sin v_0 && = \sin 5 \\
v_4 &= v_1 + v_2 && = 0.693 + 10 \\
v_5 &= v_4 - v_3 && = 10.693 + 0.959 \\
\hline
y &= v_5 && = 11.652
\end{aligned}
$$

Reverse Adjoint (Derivative) Trace

$$
\begin{aligned}
\boldsymbol{\bar{x}_1} &= \boldsymbol{\bar{v}_{-1}} && = \mathbf{5.5} \\
\boldsymbol{\bar{x}_2} &= \boldsymbol{\bar{v}_0} && = \mathbf{1.716} \\
\hline
\bar{v}_{-1} &= \bar{v}_{-1} + \bar{v}_1 \frac{\partial v_1}{\partial v_{-1}} && = \bar{v}_{-1} + \bar{v}_1 / v_{-1} && = 5.5 \\
\bar{v}_0 &= \bar{v}_0 + \bar{v}_2 \frac{\partial v_2}{\partial v_0} && = \bar{v}_0 + \bar{v}_2 \times v_{-1} && = 1.716 \\
\bar{v}_{-1} &= \bar{v}_2 \frac{\partial v_2}{\partial v_{-1}} && = \bar{v}_2 \times v_0 && = 5 \\
\bar{v}_0 &= \bar{v}_3 \frac{\partial v_3}{\partial v_0} && = \bar{v}_3 \times \cos v_0 && = -0.284 \\
\bar{v}_2 &= \bar{v}_4 \frac{\partial v_4}{\partial v_2} && = \bar{v}_4 \times 1 && = 1 \\
\bar{v}_1 &= \bar{v}_4 \frac{\partial v_4}{\partial v_1} && = \bar{v}_4 \times 1 && = 1 \\
\bar{v}_3 &= \bar{v}_5 \frac{\partial v_5}{\partial v_3} && = \bar{v}_5 \times (-1) && = -1 \\
\bar{v}_4 &= \bar{v}_5 \frac{\partial v_5}{\partial v_4} && = \bar{v}_5 \times 1 && = 1 \\
\hline
\bar{v}_5 &= \bar{y} && = 1
\end{aligned}
$$

$O(m \cdot \mathrm{ops}(f))$

Summary

- Derivatives are useful in optimization because they provide information about how to change a given point in order to improve the objective function
- For multivariate functions, various derivative-based concepts are useful for directing the search for an optimum, including the gradient, the Hessian, and the directional derivative
- computation of derivatives in computer programs can be classified into four categories:
  1. manually working out derivatives and coding them (error prone and time consuming)
  2. numerical differentiation using finite difference approximations
     Complex step method can eliminate the effect of subtractive cancellation error when taking small steps
  3. symbolic differentiation using expression manipulation in computer algebra systems
  4. automatic differentiation, (aka algorithmic differentiation)
     forward and reverse accumulation on computational graphs

3. Bracketing

Solutions and Recognizing Them for Smooth Functions

Definition

A point $\boldsymbol{x}^*$ is a global minimizer if $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x}$.

Definition

A point $\boldsymbol{x}^*$ is a local minimizer if there is a neighborhood N of $\boldsymbol{x}^*$ such that $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ for all $\boldsymbol{x} \in N$.

Solutions and Recognizing Them for Smooth Functions

Theorem (Taylor's Theorem)

Suppose that $f : \mathbb{R}^n \to \mathbb{R}$ is continuously differentiable and that $\boldsymbol{p} \in \mathbb{R}^n$. Then we have that

$$f(\boldsymbol{x} + \boldsymbol{p}) = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x} + t\boldsymbol{p})^T \boldsymbol{p},$$

for some $t \in (0, 1)$. Moreover, if f is twice continuously differentiable, we have that

$$\nabla f(\boldsymbol{x} + \boldsymbol{p}) = \nabla f(\boldsymbol{x}) + \int_0^1 \nabla^2 f(\boldsymbol{x} + t\boldsymbol{p})\boldsymbol{p} dt,$$

and that

$$f(\boldsymbol{x} + \boldsymbol{p}) = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^T \boldsymbol{p} + \frac{1}{2}\boldsymbol{p}^T \nabla^2 f(\boldsymbol{x} + t\boldsymbol{p})\boldsymbol{p},$$

for some $t \in (0, 1)$.

Solutions and Recognizing Them for Smooth Functions

Theorem (First-Order Necessary Conditions)

If $\boldsymbol{x}^*$ is a local minimizer and f is continuously differentiable in an open neighborhood of $\boldsymbol{x}^*$, then $\nabla f(\boldsymbol{x}^*) = 0$.

Definition

We call $\boldsymbol{x}^*$ a stationary point if $\nabla f(\boldsymbol{x}^*) = 0$.

Theorem (Second-Order Necessary Conditions)

If $\boldsymbol{x}^*$ is a local minimizer of f and $\nabla^2 f$ exists and is continuous in an open neighborhood of $\boldsymbol{x}^*$, then $\nabla f(\boldsymbol{x}^*) = 0$ and $\nabla^2 f(\boldsymbol{x}^*)$ is positive semidefinite.

Theorem (Second-Order Sufficient Conditions)

Suppose that $\nabla^2 f$ is continuous in an open neighborhood of $\boldsymbol{x}^*$ and that $\nabla f(\boldsymbol{x}^*) = 0$ and $\nabla^2 f(\boldsymbol{x}^*)$ is positive definite. Then $\boldsymbol{x}^*$ is a strict local minimizer of f.

Theorem

When f is convex, any local minimizer $\boldsymbol{x}^*$ is a global minimizer of f. If in addition f is differentiable, then any stationary point $\boldsymbol{x}^*$ is a global minimizer of f.

Unconstrained Optimization of Smooth Functions

Two main strategies for finding local minima of smooth functions are:

- Line search methods (gradient descent and its variants)
- Trust region methods

A component of both strategies is the ability to find a local minimum of a one-dimensional function, which is the topic of this lecture.

Bracketing: A derivative-free method to identify an interval containing a local minimum and then successively shrinking that interval

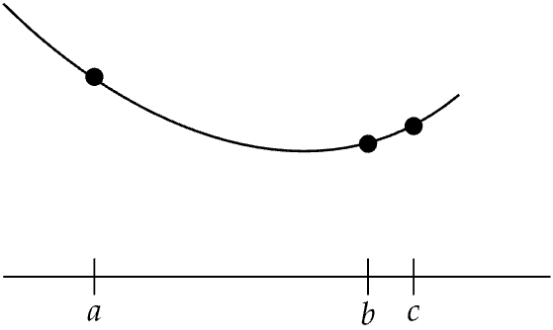
Unimodality

The algorithms in this class assume **unimodality**:

There exists a unique optimizer $\boldsymbol{x}^*$ such that f is monotonically decreasing for $\boldsymbol{x} \leq \boldsymbol{x}^*$ and monotonically increasing for $\boldsymbol{x} \geq \boldsymbol{x}^*$

Finding an Initial Bracket

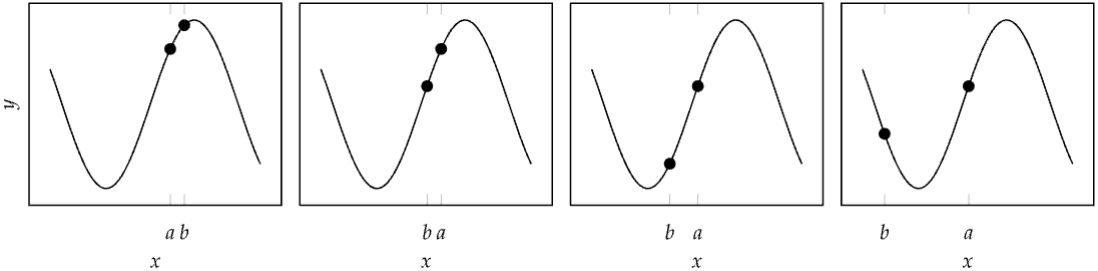
Given a unimodal function, the global minimum is guaranteed to be inside the interval $[a, c]$ if we can find three points $a < b < c$ such that $f(a) > f(b) < f(c)$




```julia
function bracket_minimum(f, x=0; s=1e-2, k=2.0)
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)
    if yb > ya
        a, b = b, a
        ya, yb = yb, ya
        s = -s
    end
    while true
        c, yc = b + s, f(b + s)
        if yc > yb
            return a < c ? (a, c) : (c, a)
        end
        a, ya, b, yb = b, yb, c, yc
        s *= k
    end
end
```

Finding an Initial Bracket

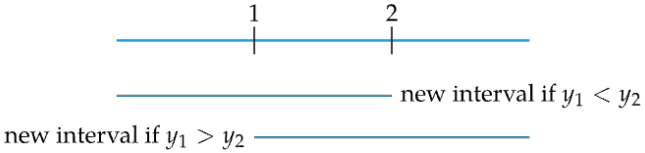
Example of `bracket_minimum` on a function



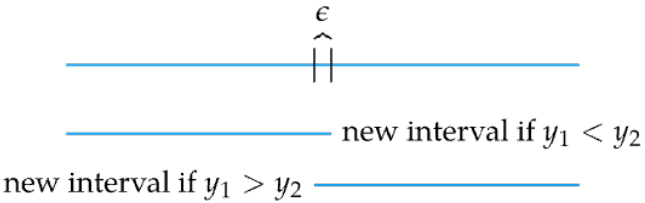
reverses direction between the first and second iteration and expands until a minimum is bracketed in the fourth iteration.

For unimodal functions, when function evaluations are limited, what is the maximal shrinkage we can achieve?

When restricted to only 2 function evaluations (queries) the most we can guarantee to shrink our interval is by just under a factor of 2.

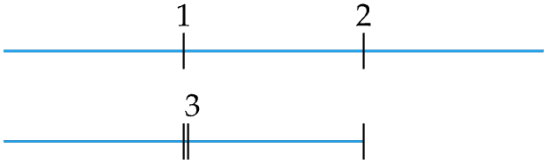


yeilds a factor of 3.



for $\epsilon \to 0$ yields a factor of just less than 2

When restricted to only 3 function evaluations (queries) the most we can guarantee to shrink our interval is by a factor of 3.

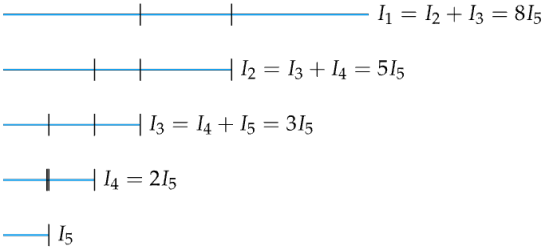


Fibonacci Search

When restricted to n functions evaluations following the previous strategy, we are guaranteed to shrink our interval by a factor of F_{n+1}.

Fibonacci numbers: sum of previous two,
$1, 1, 2, 3, 5, 8, 13, \ldots$

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1, 2 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}$$



The length of every interval constructed can be expressed in terms of the final interval times a Fibonacci number.
- final, smallest interval has length I_n,
- second smallest interval has length $I_{n-1} = F_3 I_n$
- third smallest interval has length $I_{n-2} = F_4 I_n$,

and so forth.

Fibonacci Search Algorithm

For a unimodal function f in the interval $[a, b]$, we want to shrink the interval within n iterations. (At each iteration we want to shrink by a factor ϕ).

$$b_{k+1} - a_{k+1} = \frac{F_{n-k+1}}{F_{n-k+2}}(b_k - a_k)$$

Therefore:

$$\begin{aligned} b_n - a_n &= \frac{F_2}{F_3}(b_{n-1} - a_{n-1}) \\ &= \frac{F_2}{F_3}\frac{F_3}{F_4}\cdots\frac{F_n}{F_{n+1}}(b_1 - a_1) \\ &= \frac{1}{F_{n+1}}(b_1 - a_1) \end{aligned}$$

Closed-form expression (Binet's formula):

$$F_n = \frac{\phi^n - (1-\phi)^n}{\sqrt{5}},$$

$\phi = (1+\sqrt{5})/2 \approx 1.61803$ is the golden ratio.

$$\frac{F_{n+1}}{F_n} = \phi\frac{1 - s^{n+1}}{1 - s^n}, \quad s = (1-\sqrt{5})(1+\sqrt{5}) \approx -0.38$$

Suppose we have a unimodal function f in the interval $[a, b]$ and a tolerance $\epsilon = 0.01$. Let $k = 1$.

$$\frac{F_n}{F_{n+1}} = \rho_n = \frac{1 - s^n}{\phi(1 - s^{n+1})} \approx 0.6$$

1. $d_k = a_k + \frac{F_{n-k+1}}{F_{n-k+2}}(b_k - a_k)$

2. if $k \neq n - 1$:

$$c_k = a_k + \left(1 - \frac{F_{n-k+1}}{F_{n-k+2}}\right)(b_k - a_k)$$

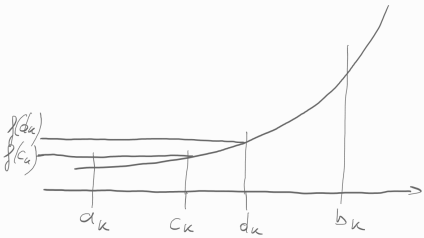
   otherwise: $c_k = d_k + \epsilon(a_k - d_k)$

3. if $f(c_k) < f(d_k)$: $\quad b_{k+1} = d_k, \quad d_{k+1} = c_k, \quad a_{k+1} = a_k$
   otherwise: $a_{k+1} = b_k, \quad b_{k+1} = c_k, \quad d_{k+1} = d_k$

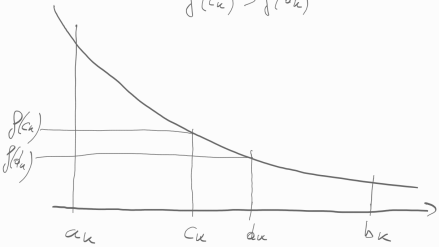
4. $k = k + 1$, if $k = n$ go to step 5, else go to step 2

5. return (a_k, b_k) if $(a_k < b_k)$ else (b_k, a_k)

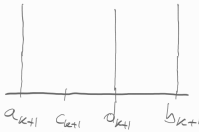
$f(c_k) < f(d_k)$



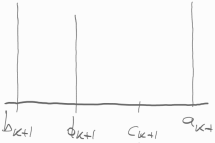
$f(c_k) > f(d_k)$



$k \neq n-1$

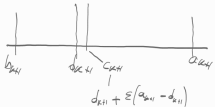


$$a_{k+1} + \underbrace{(1-\varphi)}_{\approx 0{,}4}(b_{k+1} - a_{k+1})$$



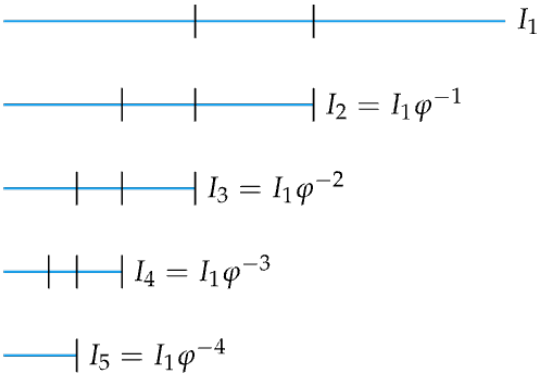
$$a_{k+1} + \underbrace{(1-\varphi)}_{\approx 0{,}4}\underbrace{(b_{k+1} - a_{k+1})}_{<0}$$

$k = n-1$

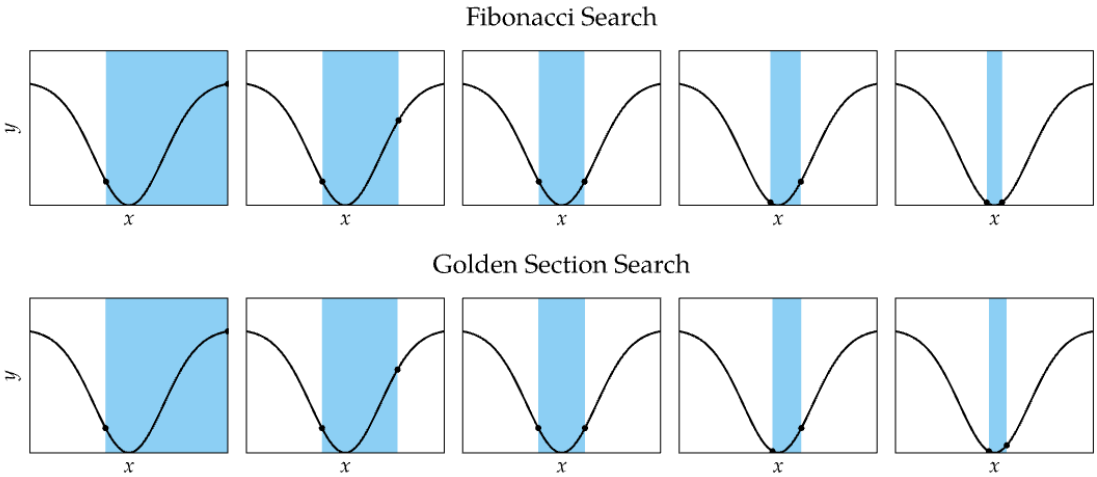


Golden Section Search

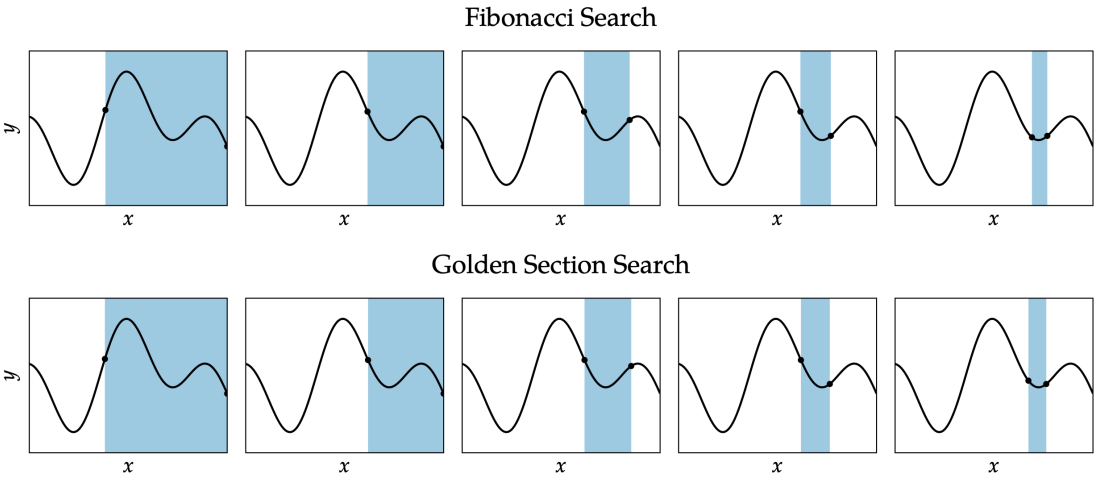
$$\lim_{n\to\infty} \frac{F_{n+1}}{F_n} = \lim_{n\to\infty} \frac{1}{\rho_n} = \lim_{n\to\infty} \phi \frac{1 - s^{n+1}}{1 - s^n} = \phi \approx 1.61803 \qquad \frac{1}{\phi} \approx 0.618$$



Comparison

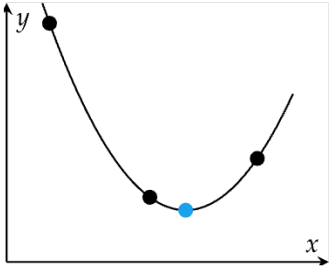


Comparison



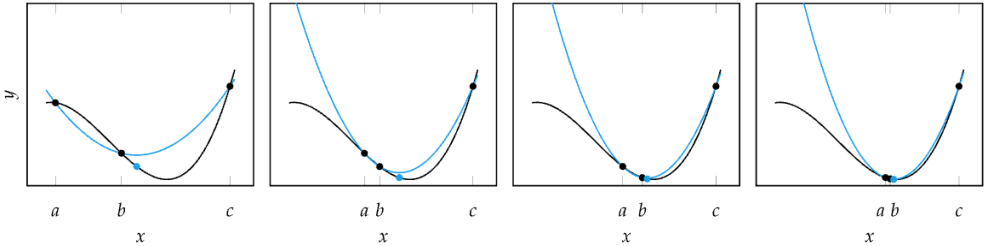
Quadratic Fit Search

- Leverages ability to analytically minimize quadratic functions
- Iteratively fits quadratic function to three bracketing points



Quadratic Fit Search

- If a function is locally nearly quadratic, the minimum can be found after several steps



Maintain a bracketing interval $[a, c]$ with $a < b < c$ and $f(a) > f(b) < f(c)$.

```
if x > b
  if yx > yb
    c, yc = x, yx
  else
    a, ya, b, yb = b, yb, x, yx
```

```
elif x < b
  if yx > yb
    a, ya = x, yx
  else
    c, yc, b, yb = b, yb, x, yx
```

Using Linear Algebra

- We assume that the variable y is related to $x \in \mathbb{R}$ quadratically, so for some constants b_0, b_1, b_2:

$$y = b_0 + b_1 x + b_2 x^2$$

- Given the set of 3 points $(y_1, x_1,), \ldots, (y_3, x_3)$ in the ideal case, we have that $y_i = b_0 + b_1 x_i + b_2 x_i^2$, for all $i = 1, 2, 3$. In matrix form:

$$\begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

This can be written as $A\boldsymbol{z} = \boldsymbol{y}$ to emphasize that $\boldsymbol{z}$ are our unknowns and A and $\boldsymbol{y}$ are given.

In Python

In polynomial regression, the $m \times (n+1)$ matrix A is called a **Vandermonde matrix** (a matrix with entries $a_{ij} = x_i^{n+1-j}$, $j = 1..n+1$).
NumPy's `np.vander()` is a convenient tool for quickly constructing a Vandermonde matrix, given the values $x_i, i = 1..m$, and the number of desired columns $(n+1)$.

```
>>> print(np.vander([2, 3, 5], 2))
[[2 1]                              # [[2**1, 2**0]
 [3 1]                              #  [3**1, 3**0]
 [5 1]]                             #  [5**1, 5**0]]

>>> print(np.vander([2, 3, 5, 4], 3))
[[ 4  2  1]                         # [[2**2, 2**1, 2**0]
 [ 9  3  1]                         #  [3**2, 3**1, 3**0]
 [25  5  1]                         #  [5**2, 5**1, 5**0]
 [16  4  1]]                        #  [4**2, 4**1, 4**0]
```

In Python

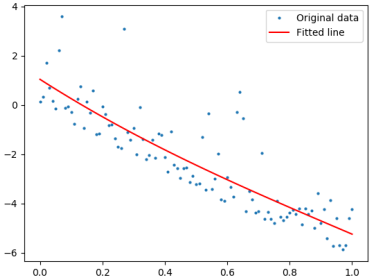
```
A = np.vander(x,4)

coeff = np.linalg.solve(A,y) ## Error!! Why?

B = A.T @ A
z = np.linalg.inv(B) @ A.T @ y

coeff = np.linalg.lstsq(A, y)[0]
np.allclose(z,coeff)

f=np.poly1d(coeff)
plt.plot(x, y, 'o', label='Original data', ↪
    ↪markersize=2)
plt.plot(x, f(x), 'r', label='Fitted line')
plt.legend()
plt.show()
```



However, for three points the system $Az = \boldsymbol{y}$ is solvable, also in closed form!

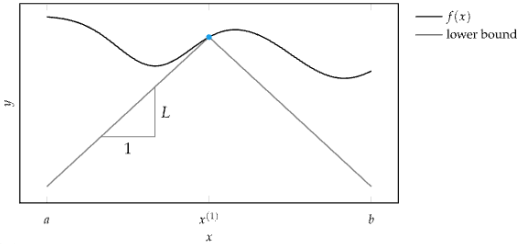
Shubert-Piyavskii Method

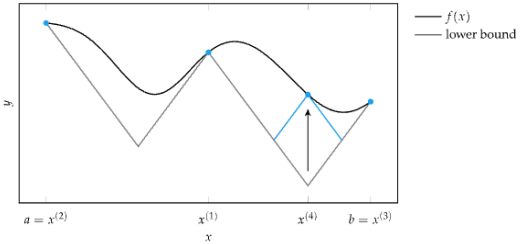
- The Shubert-Piyavskii method is guaranteed to find the global minimum of any bounded function

- but requires that the function be Lipschitz continuous

- A function is **Lipschitz continuous** if there is an upper bound on the magnitude of its derivative. A function f is Lipschitz continuous on $[a, b]$ if there exists an $\ell > 0$ such that:

$$|f(x) - f(y)| \leq \ell|x - y|, \quad \forall x, y \in [a, b]$$





The Algorithm

Given a valid Lipschitz constant ℓ and an initial interval $[a, b]$:

<u>Initialize:</u>
Sample the midpoint, $x^{(1)} = (a + b)/2$.
Construct a sawtooth lower bound using lines of slope $\pm\ell$ from $x^{(1)}$. These lines will always lie below f if ℓ is a valid Lipschitz constant

<u>Iterate:</u>
Upper vertices in the sawtooth correspond to sampled points. Lower vertices correspond to intersections between the Lipschitz lines originating from each sampled point.
Further iterations find the minimum point in the sawtooth $(x^{(i)}, y^{(i)})$, evaluate the function at that $x^{(i)}$ value, $f(x^{(i)})$, and then use the result to update the sawtooth.

<u>Terminate:</u>

when: $|y^{(i)} - f(x^{(i)})| < \epsilon$.

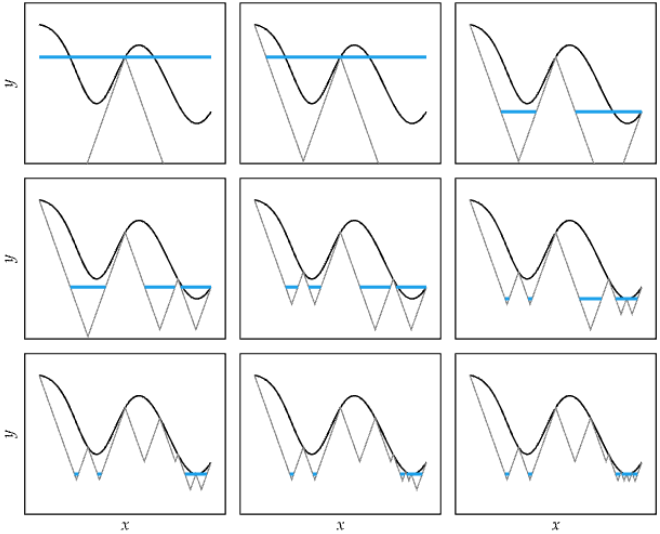
For every peak, an uncertainty region (blue segments) can be computed according to:

$$\left[x^{(i)} - \frac{1}{\ell}\left(y_{\min} - y^{(i)}\right),\right.$$

$$\left.x^{(i)} + \frac{1}{\ell}\left(y_{\min} - y^{(i)}\right)\right]$$

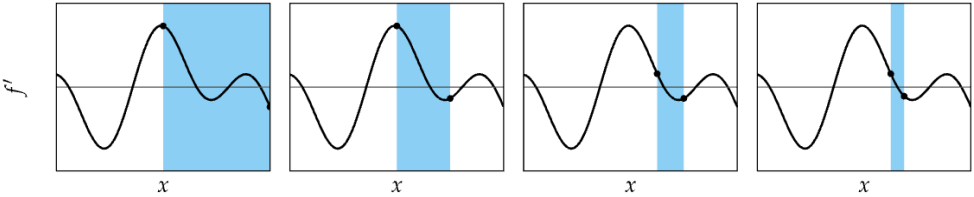
$\left(x^{(i)}, y^{(i)}\right)$ sawtooth lower vertex

$(x_{\min}, y_{\min})$ minimum sawtooth upper vertex



Bisection Method

- **Intermediate value theorem**: If f is continuous on $[a, b]$, and there is some $y \in [f(a), f(b)]$, then there exists at least one $x \in [a, b]$, such that $f(x) = y$.
- Used in root-finding methods
- When applied to $f'(x)$, can be used to find minimum of f
- if $\text{sign}(f'(a)) \neq \text{sign}(f'(b))$, or equivalently, $f'(a)f'(b) \leq 0$ then $[a, b]$ is guaranteed to contain a zero.



Bisection Method

- Cut the bracketed region $[a, b]$ in half with every iteration
- Evaluate the midpoint $(a + b)/2$
- form a new bracket from the midpoint and whichever side that continues to bracket a zero.
- Terminate after a fixed number of iterations.
- Guaranteed to converge within ϵ of x^* within $\lg_2(|b - a|/\epsilon)$

Summary

- Many optimization methods shrink a bracketing interval, including Fibonacci search, golden section search, and quadratic fit search
- The Shubert-Piyavskii method outputs a set of bracketed intervals containing the global minima, given the Lipschitz constant
- Root-finding methods like the bisection method can be used to find where the derivative of a function is zero

4. Local Descent

Preface

For multivariate functions, we have argued that:

- derivatives can have exponential growth in the resulting analytical expression
- calculating zeros might be challenging

Hence, minimizing by solving $\nabla f(\boldsymbol{x}) = 0$ may be computationally demanding.

Outline

7. Line Search Methods

8. Convergence Analysis

9. Trust Region Methods

Descent Direction Iteration

Descent Direction Methods use a local model to incrementally improve design point until some convergence criteria is met.

1. Check termination conditions at $\boldsymbol{x}_k$; if not met, continue.
2. Decide **descent direction** $\boldsymbol{d}_k$ using local information, commonly required that $\boldsymbol{d}_k^T \nabla f(\boldsymbol{x}_k) < 0$.
3. Decide **step size** (ie, magnitude of the overall step that depends on α_k, sometimes but not always $\|\boldsymbol{d}_k\|_2 = 1$)
4. Compute next design point $\boldsymbol{x}_{k+1}$

$$\boxed{\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k}$$

Descent Direction

The search direction often has the form

$$\boldsymbol{d}_k = -B_k^{-1} \nabla f(\boldsymbol{x}_k)$$

where B_k is a symmetric and nonsingular matrix.

- in the steepest descent method, B_k is the identity matrix I
- in Newton's method, B_k is the exact Hessian $\nabla^2 f(\boldsymbol{x}_k)$.
- in quasi-Newton methods, B_k is an approximation to the Hessian that is updated at every iteration by means of a low-rank formula.

When $\boldsymbol{d}_k = -B_k^{-1} \nabla f(\boldsymbol{x}_k)$ and B_k is positive definite, we have

$$\boldsymbol{d}_k^T \nabla f(\boldsymbol{x}_k) = -\nabla f(\boldsymbol{x}_k)^T B_k^{-1} \nabla f(\boldsymbol{x}_k) < 0$$

and therefore $\boldsymbol{d}_k$ is a descent direction. In fact, it is a double implication!

Outline

- We discuss how to choose α_k and $\boldsymbol{d}_k$ to promote convergence from remote starting points.
- We also consider the rate of convergence of steepest descent, quasi-Newton, and Newton methods.

Line Search for Step Size

Assuming we have the search direction:

- Use it to compute α
- Using the techniques discussed in the previous class, find the minimum of a univariate function:

$$\phi(\alpha) = f(\boldsymbol{x} + \alpha \boldsymbol{d}) \qquad \text{minimize}_{\alpha \geq 0}\ \phi(\alpha)$$

We assume that $\boldsymbol{p}_k$ is a descent direction, that is, $\phi'(0) < 0$, so that our search can be confined to positive values of α.

```
def line_search(f, x, d)
  objective = lambda alpha: f(x + alpha * d)
  a, b = bracket_minimum(objective)
  alpha = minimize(objective, a, b)
  return x + alpha * d
```

Often computationally costly, so approximate line search is used instead

Line Search: Alternatives

α is called to the **learning rate** or **step factor**:

Equal to the **step size** only when $\|\boldsymbol{d}_k\|_2 = 1$.

- Fixed learning rate
- **Decaying step factor**

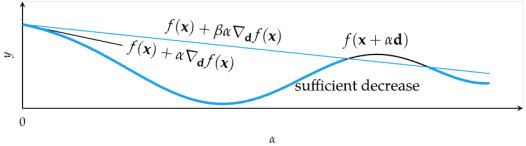
$$\alpha_k = \alpha_1 \gamma^{k-1} \qquad \text{for } \gamma \in [0, 1]$$

Decaying step factor is often required in convergence proofs

Approximate Line Search

- If function calls are expensive, rather than finding the minimum along a search direction, find a point of sufficient decrease

$$f(\boldsymbol{x}_{k+1}) \leq f(\boldsymbol{x}_k) + \beta\alpha\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$$

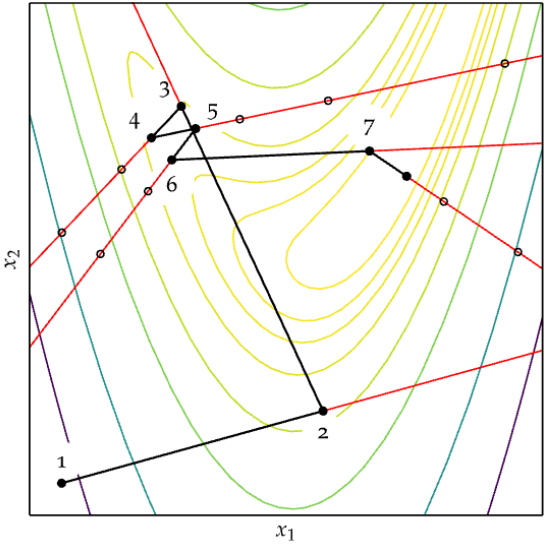


- $\beta \in [0, 1]$, usually $\beta = 1 \times 10^{-4}$
- Alone this condition is insufficient to guarantee convergence to a local minimum. It can converge prematurely.
- Backtracking line search starts with a large step and then backs off

```
def backtracking_line_search(f, grad, x, d, alpha_0=1, p=0.5, beta=1e-4):
    y, g, alpha = f(x), grad(x), alpha_0
    while ( f(x + alpha * d) > y + beta * alpha * np.dot(g, d) ) :
        alpha *= p
    return alpha
```

- Guaranteed to converge to a local minimum, but can be slow.

Approximate Line Search: Example



Approximate Line Search

Building on backtracking line search are the **Wolfe Conditions** together sufficient to guarantee convergence to a local minimum.

1. First Wolfe Condition: Sufficient Decrease (Armijo condition)

$$f(\boldsymbol{x}_{k+1}) \leq f(\boldsymbol{x}_k) + \beta\alpha\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$$

2. Second Wolfe Condition: Curvature Condition

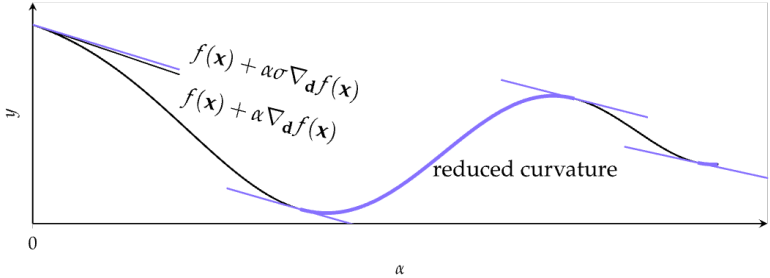
$$\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1}) \geq \sigma\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$$

$\beta < \sigma < 1$ with
- $\sigma = 0.1$ with conjugate gradient method
- $\sigma = 0.9$ with Newton method

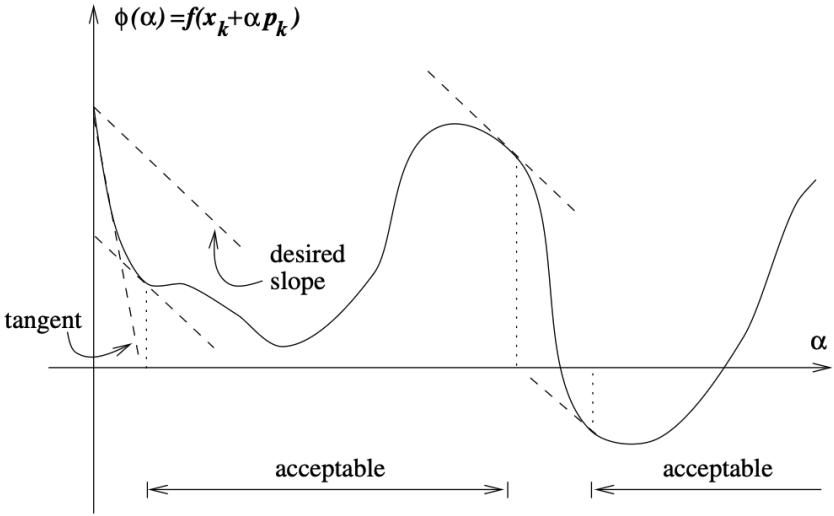
Curvature Condition

Regions satisfying the curvature condition $\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1}) \geq \sigma \nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)$

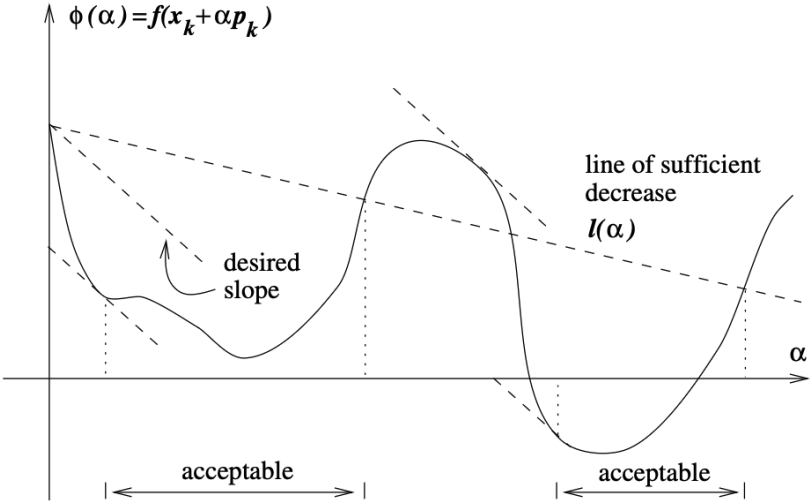
- Consider the univariate function $\phi(\alpha) = f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$.
- The left-hand-side is simply the derivative $\phi'(\alpha_k)$, so the curvature condition ensures that the slope of ϕ at α_k is greater than σ times the initial slope $\phi'(0)$.
- If the slope $\phi'(0)$ is negative, then we are looking for a step size α_k such that the slope of ϕ at that point is still negative but not too negative.
- If $\phi'(\alpha_k)$ is only slightly negative or even positive, it is a sign that we cannot expect much more decrease in f in this direction, so it makes sense to terminate the line search.



Curvature Condition



Wolfe Conditions



Approximate Line Search: Example

Consider approximate line search on $f(x_1, x_2) = x_1^2 + x_1 x_2 + x_2^2$
from $\boldsymbol{x} = [1, 2]$ in the direction $\boldsymbol{d} = [-1, -1]$, gradient at $\boldsymbol{x}$ is $\boldsymbol{g} = [4, 5]$
using a maximum step size of 10, a reduction factor of 0.5,
first Wolfe condition parameter $\beta = 1 \times 10^{-4}$, second Wolfe condition parameter $\sigma = 0.9$.

First Wolfe condition ($f(\boldsymbol{x} + \alpha \boldsymbol{d}) \leq f(\boldsymbol{x}) + \beta\alpha(\boldsymbol{g}^T \cdot \boldsymbol{d})$):

$$\begin{aligned}
\alpha = 10: &\quad f([1,2] + 10 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 10 [4,5]^T [-1,-1] \implies 217 \not\leq 6.991 \\
\alpha = 10 \cdot 0.5 = 5: &\quad f([1,2] + 5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 5 [4,5]^T [-1,-1] \implies 37 \not\leq 6.996 \\
\alpha = 2.5: &\quad f([1,2] + 2.5 \cdot [-1,-1]) \leq 7 + 1 \times 10^{-4} 2.5 [4,5]^T [-1,-1] \implies 3.25 \leq 6.998
\end{aligned}$$

The candidate design point $\boldsymbol{x}' = \boldsymbol{x} + \alpha \boldsymbol{d} = [-1.5, -.0.5]$ is checked against the second Wolfe condition $\nabla_{\boldsymbol{d}} f(\boldsymbol{x}') \geq \sigma \nabla_{\boldsymbol{d}} f(\boldsymbol{x})$:

$$[-3.5, -2.5] \cdot [-1, -1] \geq \sigma [4, 5] \cdot [-1, -1] \implies 6 \geq -8.1$$

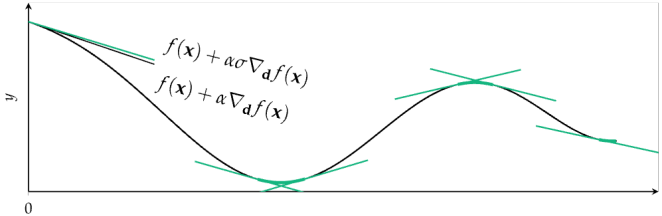
Approximate line search terminates with $\boldsymbol{x} = [-1.5, -0.5]$.

Strong Wolfe Conditions

A step length may satisfy the Wolfe conditions without being particularly close to a minimizer of $f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$.
We can modify the curvature condition to force $f(\boldsymbol{x}_{k+1})$ to exclude points that are far from stationary points of $f(\boldsymbol{x}_{k+1})$, ie, we no longer allow the gradient $\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1})$ to be too positive.

Strong Wolf conditions:

$$f(\boldsymbol{x}_{k+1}) \leq f(\boldsymbol{x}_k) + \beta\alpha\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k) \qquad \text{and} \qquad |\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_{k+1})| \leq \sigma|\nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k)|$$



Approximate Line Search Goal

Given:

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$$

with descent direction:

$$\nabla f(\boldsymbol{x}_k)^T \boldsymbol{d}_k < 0$$

Define:

$$\phi(\alpha) = f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k), \qquad \phi'(\alpha) = \nabla f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)^T \boldsymbol{d}_k$$

Goal:

Find $\alpha > 0$ satisfying the **Strong Wolfe Conditions**.

Strong Wolfe Conditions

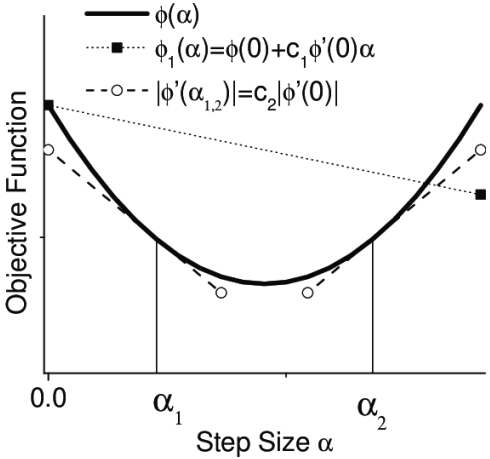
Two requirements:

Sufficient decrease (Armijo)

$$\phi(\alpha) \leq \phi(0) + \beta\alpha\phi'(0)$$

Strong curvature condition

$$|\phi'(\alpha)| \leq \sigma|\phi'(0)|$$



Ensures:

- sufficient decrease
- step near minimum

Strong Backtracking: Idea

Two phases:

1. **Bracketing phase**
   - Start with $[\alpha_0 = 0, \alpha_1]$
   - Increase step
   - Find interval containing solution

2. **Zoom phase**
   - Shrink interval
   - Use bisection or interpolation
   - Find Wolfe step

Idea:

Bracket minimum $\rightarrow$ Zoom to solution

Algorithm

Bracketing

- Start with $[\alpha_0 = 0, \alpha_1]$
- If Armijo fails or slope changes sign

  $\rightarrow$ Zoom

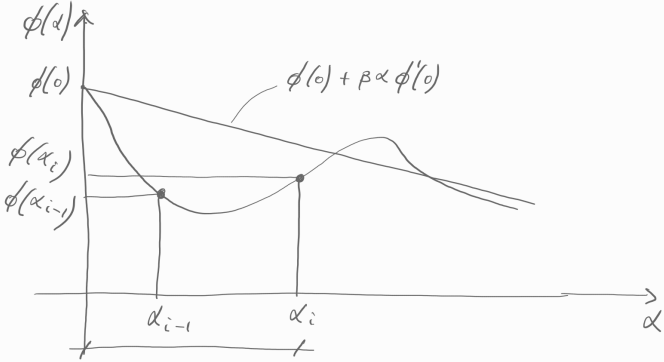
- If strong Wolfe satisfied

  $\rightarrow$ Stop

- Else increase step

Input : α_{max} maximum step allowed
Output: α^* set to a step length that satisfies the strong Wolfe condition

Initialize $k \leftarrow 0$;
Set $\alpha_0 \leftarrow 0$, choose $\alpha_{max} > 0$ and $\alpha_1 \in (0, \alpha_{max})$;
$i \leftarrow 1$;
while True **do**
  Evaluate $\phi(\alpha_i)$;
  if $\phi(\alpha_i) > \phi(0) + \beta\alpha_i\phi'(0)$ or $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ and $i > 1]$ **then** // Armijo or next slide
    $\alpha^* \leftarrow \text{zoom}(\alpha_{i-1}, \alpha_i)$ and break;
  Evaluate $\phi'(\alpha_i)$;
  if $|\phi'(\alpha_i)| \leq -\sigma\phi'(0)$ **then**
    set $\alpha^* \leftarrow \alpha_i$ and break;
  if $\phi'(\alpha_i) \geq 0$ **then** // slope changes
    set $\alpha^* \leftarrow \text{zoom}(\alpha_i, \alpha_{i-1})$ and break;
  Choose $\alpha_{i+1} \in (\alpha_i, \alpha_{max})$; // Eg, $\alpha_{i+1} \leftarrow 2\alpha_i$
  $i \leftarrow i + 1$;


$\phi(\alpha)$
$\phi(0)$
$\phi(0) + \beta\alpha\phi'(0)$
$\phi(\alpha_i)$
$\phi(\alpha_{i-1})$
α_{i-1}
α_i
α

Algorithm

Zoom

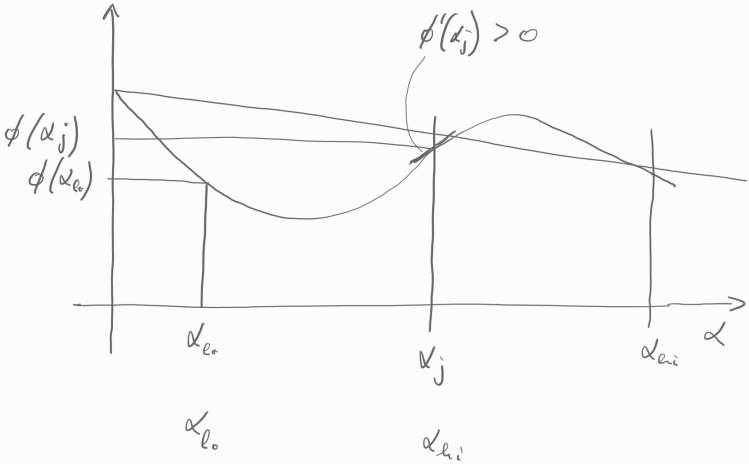
- Interpolate inside bracket
- Shrink interval
- Stop when Wolfe satisfied

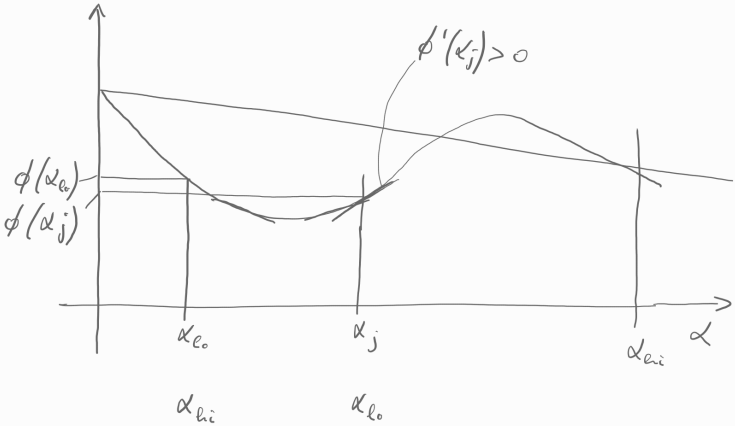
Iterate generating an $\alpha_j \in [\alpha_{lo}, \alpha_{hi}]$, and then replacing one of these endpoints by α_j maintaining the invariants:

1. $[\alpha_{lo}, \alpha_{hi}]$ satisfy the strong Wolfe conditions;
2. α_{lo} smallest function value $\phi(\alpha_{lo})$;
3. α_{hi} chosen so that $\phi'(\alpha_{lo})(\alpha_{hi} - \alpha_{lo}) < 0$.

Input : α_{lo}, α_{hi}
Output: α^* set to a step length that satisfies the strong Wolfe condition

```
while True do
    Interpolate (using quadratic, cubic or bisection) to find a trial step
      length α_j between α_lo and α_hi ;
    Evaluate φ(α_j);
    if φ(α_j) > φ(0) + βα_j φ'(0) or φ(α_j) ≥ φ(α_lo) then // next slide
        α_hi ← α_j;
    else
        Evaluate φ'(α_j);
        if |φ'(α_j)| ≤ −σφ'(0) then
            Set α* ← α_j and break;
        if φ'(α_j)(α_hi − α_lo) ≥ 0 then // violates 3., second next slide
            α_hi ← α_lo;
        α_lo ← α_j;
```


$\phi'(\alpha_j) > 0$
$\phi(\alpha_j)$
$\phi(\alpha_{l_0})$
α_{l_0}
α_j
α_{l_i}
α
α_{l_0}
α_{l_i}


$\phi'(\alpha_j) > 0$
$\phi(\alpha_{l_0})$
$\phi(\alpha_j)$
α_{l_0}
α_j
$\alpha_{l_{hi}}$
α
$\alpha_{l_{hi}}$
α_{l_0}

Approximate Line Search: Python Implementation

```
def strong_backtracking(f, nabla, x, d; alpha=1, beta=1e-4, sigma=0.1)
  y0, g0, y_prev, alpha_prev = f(x), nabla(x) @ d, NaN, 0
  alpha_lo, alpha_hi = NaN, NaN
```

```
  # bracket phase
  while true
    y = f(x + alpha*d)
    if y > y0 + beta*alpha*g0 || (!isnan(y_prev) &&↪
        ↪ y >= y_prev)
      alpha_lo, alpha_hi = alpha_prev, alpha
      break
    g = nabla(x + alpha*d) @ d
    if abs(g) <= -sigma * g0
      return alpha
    elif g <= 0
      alpha_lo, alpha_hi = alpha, alpha_prev
      break
    y_prev, alpha_prev, alpha = y, alpha, 2 * alpha
```

```
  # zoom phase
  ylo = f(x + alpha_lo*d)
  while true:
    alpha = (alpha_lo + alpha_hi)/2
    y = f(x + alpha*d)
    if y > y0 + beta*alpha*g0 || y >= ylo
      alpha_hi = alpha
    else
      g = nabla(x + alpha*d) @ d
      if abs(g) <= -sigma*g0
        return alpha
      elif g*(alpha_hi -alpha_lo) >= 0
        alpha_hi = alpha_lo
      alpha_lo = alpha
```

Why Strong Backtracking?

Used in:

- Gradient descent
- Quasi-Newton: BFGS, L-BFGS
- Nonlinear Conjugate Gradient

Guarantees:

- global convergence
- stable steps
- fast convergence of quasi-Newton

Standard method in modern optimization.

Note: it is an algorithm that makes use of derivative information!

Outline

7. Line Search Methods

8. Convergence Analysis

9. Trust Region Methods

Convergence Analysis

Let $\{\boldsymbol{x}_k\}$ be a sequence of points belonging in $\mathbb{R}^n$.
We say that a sequence $\{\boldsymbol{x}_k\}$ **converges** to some point $\boldsymbol{x}$, written $\lim_{k\to\infty} \boldsymbol{x}_k = x$, if for any $\epsilon > 0$, there is an index K such that

$$\|\boldsymbol{x}_k - \boldsymbol{x}\| \leq \epsilon \qquad \text{for all } k \geq K$$

For example, the sequence $\boldsymbol{x}_k$ defined by $\boldsymbol{x}_k = (1 - 2^{-k}, 1/k^2)^T$ converges to $(1, 0)^T$.

Convergence of Line Search

Let θ_k be the angle between $\boldsymbol{d}_k$ and the steepest descent direction $-\nabla f_k$, defined by:

$$\cos\theta_k = \frac{-\nabla f_k^T \boldsymbol{d}_k}{\|\nabla f_k\| \, \|\boldsymbol{d}_k\|}$$

Theorem (Zoutendijk condition)

Consider any iteration of the form $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$, where $\boldsymbol{d}_k$ is a descent direction and α_k satisfies the Wolfe conditions. Suppose that f is bounded below in $\mathbb{R}^n$ and that f is continuously differentiable in an open set N containing the level set $\mathcal{L} = \{x : f(x) \leq f(x_0)\}$, where $\boldsymbol{x}_0$ is the starting point of the iteration. Assume also that the gradient ∇f is Lipschitz continuous on N, that is, there exists a constant $L > 0$ such that:

$$|f(x) - f(y)| \leq L|x - y|, \quad \forall x, y \in N$$

Then:

$$\sum_{k \geq 0} \cos^2\theta_k \, \|\nabla f_k\|^2 < \infty$$

The Zoutendijk condition implies that

$$\cos^2 \theta_k \, \|\nabla f_k\|^2 \to 0$$

We can be sure that the gradient norms $\|\nabla f_k\|$ converge to zero, provided that the search directions are never too close to orthogonality with the gradient.

It is a **global convergence** result.
Here the strongest possible result: we cannot guarantee that the method converges to a minimizer, but only that it is attracted by stationary points (only introducing the Hessian we can prove convergence to a local minimum — see next slide)

In

$$\boldsymbol{d}_k = -B_k^{-1} \nabla f(\boldsymbol{x}_k)$$

assume that the matrices B_k are **positive definite** with a uniformly bounded condition number. That is, there is a constant M such that $\|B_k\| \left\|B_k^{-1}\right\| \leq M$ for all k.

Then from the definition of θ_k:

$$\cos\theta_k \geq 1/M$$

By combining this bound with $\cos^2\theta_k \|\nabla f_k\|^2 \to 0$ we find that

$$\lim_{k\to\infty} \|\nabla f_k\| = 0$$

So, globally convergent under the positive definitness assumptions on B_k, (which is needed to ensure that $\boldsymbol{d}_k$ is a descent direction), and if the step lengths satisfy the Wolfe conditions.

Rate of Convergence

Let $\{\boldsymbol{x}_k\}$ be a sequence in $\mathbb{R}^n$ that converges to $\boldsymbol{x}^*$.
The convergence is said to be **Q-linear** (quotient-linear) if there is a constant $r \in (0, 1)$ such that

$$\boxed{\frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|} \leq r \qquad \text{for all } k \text{ sufficiently large}}$$

ie, the distance to the solution $\boldsymbol{x}^*$ decreases at each iteration by at least a constant factor bounded away from 1 (ie, < 1).

Example:
sequence $\{1 + (0.5)^k\}$ converges Q-linearly to 1, with rate $r = 0.5$.

Rate of Convergence

The convergence is said to be **Q-superlinear** if

$$\boxed{\lim_{k\to\infty} \frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|} = 0}$$

Example: the sequence $\{1 + k^{-k}\}$ converges superlinearly to 1.

An even more rapid convergence rate: The convergence is said to be **Q-quadratic** if

$$\boxed{\frac{\|\boldsymbol{x}_{k+1} - \boldsymbol{x}^*\|}{\|\boldsymbol{x}_k - \boldsymbol{x}^*\|^2} \leq M \qquad \text{for all } k \text{ sufficiently large}}$$

where M is a positive constant, not necessarily less than 1. Example: the sequence $\{1 + (0.5)^{2^k}\}$. The values of r and M depend not only on the algorithm but also on the properties of the particular problem. Regardless of these values a quadratically convergent sequence will always eventually converge faster than a linearly convergent sequence.

Rate of Convergence

Superlinear convergence (quadratic, cubic, quartic, etc) is regarded as fast and desirable, while sublinear convergence is usually impractical.

- Steepest descent algorithms converge only at a Q-linear rate, and when the problem is ill-conditioned the convergence constant r is close to 1.
- Quasi-Newton methods for unconstrained optimization typically converge Q-superlinearly
- Newton's method converges Q-quadratically under appropriate assumptions.

Rate of Convergence

A slightly weaker form of convergence:
overall rate of decrease in the error, rather than the decrease over each individual step of the algorithm.
We say that convergence is **R-linear** (root-linear) if there is a sequence of nonnegative scalars $\{v_k\}$ such that

$$\|\boldsymbol{x}_k - \boldsymbol{x}^*\| \leq \{v_k\} \text{ for all } k, \text{ and } \{v_k\} \text{ converges Q-linearly to zero.}$$

Outline

7. Line Search Methods

8. Convergence Analysis

9. Trust Region Methods

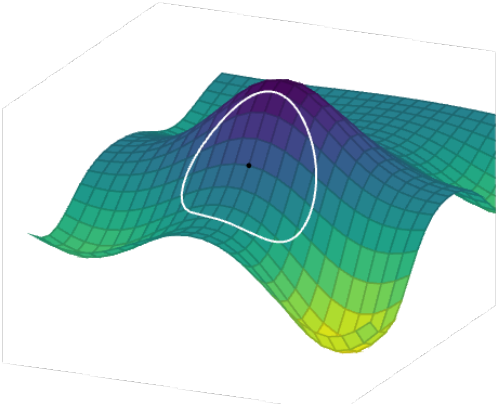
Trust Region Methods

- Descent methods can place too much trust in their first and second order information
- A **trust region** is the local area of the design space where the local model is believed to be reliable.
- Trust region methods, or restricted step methods, limit the step size to ensure local approximation error is minimized
- If the improvement matches the predicted value, the trust region is expanded; otherwise it is contracted

Trust Region Methods

- $\boldsymbol{x}'$ is new design point
- $\hat{f}(\boldsymbol{x}')$ is local function approximation, eg, second-order Taylor approximation
- δ is trust region radius

$$\begin{aligned} &\text{minimize}_{\boldsymbol{x}'}\ \hat{f}(\boldsymbol{x}') \\ &\text{subject to} \quad ||\boldsymbol{x} - \boldsymbol{x}'|| \leq \delta \end{aligned}$$



Constrained optimization problem.
It can be solved efficiently if $\hat{f}$ quadratic

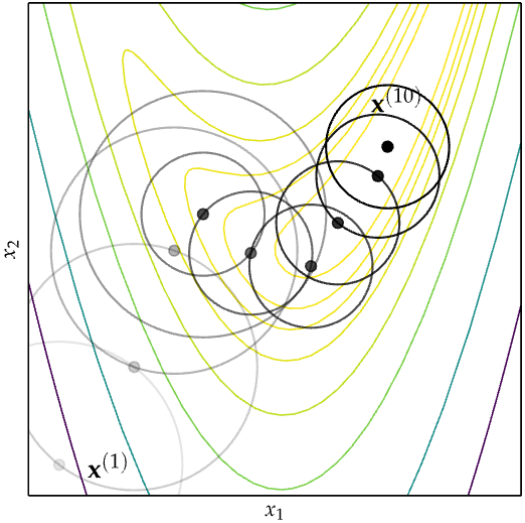
Trust Region Methods

δ can be expanded or contracted based on performance

$$\eta = \frac{\text{actual improvement}}{\text{predicted improvement}} = \frac{f(\boldsymbol{x}) - f(\boldsymbol{x}')}{f(\boldsymbol{x}) - \hat{f}(\boldsymbol{x}')}$$

If $\eta < \eta_1$ contract by a factor $\gamma_k < 1$
if $\eta > \eta_2$ expand by a factor $\gamma_k > 1$

Trust Region Methods: Example



Trust regions can be also non circular.

Termination Conditions

For all iterative descent methods and commonly used together:

- Maximum Iterations: $k > k_{\max}$
- Absolute Improvement: $f(\boldsymbol{x}_k) - f(\boldsymbol{x}_{k+1}) < \epsilon_a$
- Relative Improvement: $f(\boldsymbol{x}_k) - f(\boldsymbol{x}_{k+1}) < \epsilon_r |f(\boldsymbol{x}_k)|$
- Gradient Magnitude: $||\nabla f(\boldsymbol{x}_{k+1})|| < \epsilon_g$

Then random restart.

Summary

- Descent direction methods incrementally descend toward a local optimum.
- Univariate optimization can be applied during line search.
- Approximate line search can be used to identify appropriate descent step sizes.
- Trust region methods constrain the step to lie within a local region that expands or contracts based on predictive accuracy.
- Termination conditions for descent methods can be based on criteria such as the change in the objective function value or magnitude of the gradient.

5. First-Order Methods

Descent Direction Methods

How to select the descent direction?

- first-order methods that rely on gradient
- second-order methods that rely on Hessian information

Advantages of first order methods:

- cheap iterations: good for small and large scale optimization
- helpful because easy to warm restart

Limitations of first order methods:

- not hard to find challenging instances for them (instances with many saddle regions).
- can converge slowly.

Outline

10. Gradient Descent

11. Conjugate Descent

12. Accelerated Descents

Gradient Descent

The **steepest descent** direction at $\boldsymbol{x}_k$, at kth iteration of a **local descent iterative method**, is the one opposite to the gradient (**gradient descent**):

$$\boxed{\boldsymbol{d}_k = -\frac{\nabla f(\boldsymbol{x}_k)}{\|\nabla f(\boldsymbol{x}_k)\|}}$$

Guaranteed to lead to improvement if:

- f is smooth
- step size is sufficiently small
- $\boldsymbol{x}_k$ is not a stationary point (ie, $\nabla f(\boldsymbol{x}_k) = 0$)

Gradient Descent: Example

- Suppose we have

$$f(\boldsymbol{x}) = x_1 x_2^2$$

- The gradient is $\nabla f = [x_2^2, 2x_1x_2]$
- $\boldsymbol{x}_k = [1, 2]$

$$\boldsymbol{d}_{k+1} = -\frac{\nabla f(\boldsymbol{x}_k)}{\|\nabla f(\boldsymbol{x}_k)\|} = \frac{[-4, -4]}{\sqrt{16+16}} = \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right]$$

Implementation

```python
class DescentMethod:
    alpha: float

class GradientDescent(DescentMethod):
    def __init__(self, f, grad, x, alpha):
        self.alpha = alpha

    def step(self, f, grad, x):
        alpha, g = self.alpha, grad(x)
        return x - alpha * g
```

Gradient Descent

<u>Theorem:</u> The next direction is orthogonal to the current direction.

<u>Proof:</u>

$$\alpha_k^* = \underset{\alpha}{\operatorname{argmin}}\ f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$$

$$\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k) = \nabla_{\boldsymbol{d}_k} f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k) = 0$$ because α_k^* is minimum

$$\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)^T \boldsymbol{d}_k = 0$$ because directional derivative: $\nabla_{\boldsymbol{s}} f(\boldsymbol{x}) = \nabla f(\boldsymbol{x})^T \boldsymbol{s}$

$$\boldsymbol{d}_{k+1} = -\frac{\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)}{\|\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)\|}$$ gradient descent

$$\boldsymbol{d}_{k+1} \cdot \boldsymbol{d}_k = -\frac{\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)}{\|\nabla f(\boldsymbol{x}_k + \alpha_k^* \boldsymbol{d}_k)\|} \cdot \boldsymbol{d}_k = 0$$ $\boldsymbol{d}_{k+1}^T \boldsymbol{d}_k = 0 \implies \boldsymbol{d}_{k+1} \perp \boldsymbol{d}_k$

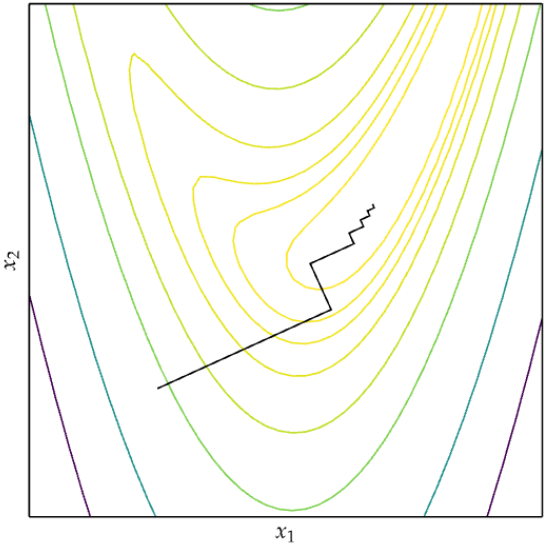
$\square$

Gradient Descent: Example

2D Rosenbrock function

$$f(x, y) = (a - x)^2 + b(y - x^2)^2$$

Narrow valleys not aligned with gradient can be a problem



Outline

10. Gradient Descent

11. Conjugate Descent

12. Accelerated Descents

Conjugate Gradient

[Hestenes and Stiefel, 1950s]

For A symmetric positive definite:

$$A\boldsymbol{x} = \boldsymbol{b} \iff \underset{\boldsymbol{x}}{\text{minimize}}\ f(\boldsymbol{x}) \stackrel{def}{=} \frac{1}{2}\boldsymbol{x}^T A\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}$$

$$\nabla f(\boldsymbol{x}) = A\boldsymbol{x} - \boldsymbol{b} \stackrel{def}{=} \boldsymbol{r}(\boldsymbol{x})$$

Conjugate Direction

Def.: A set of nonzero vectors $\{\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_\ell\}$ is said to be **conjugate** with respect to the symmetric positive definite matrix A if

$$\boldsymbol{d}_i^T A \boldsymbol{d}_j = 0, \qquad \text{for all } i \neq j$$

(the vectors are linearly independent. Generally, not orthogonal.)

Theorem: Given an arbitrary $\boldsymbol{x}_0 \in \mathbb{R}^n$ and a set of conjugate vectors $\{\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_{n-1}\}$ the sequence $\{\boldsymbol{x}_k\}$ generated by

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$$

where α_k is the analytical solution of $\min_\alpha f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$ given by:

$$\alpha_k = -\frac{\boldsymbol{r}_k^T \boldsymbol{d}_k}{\boldsymbol{d}_k^T A \boldsymbol{d}_k}$$

(aka, **conjugate direction algorithm**) converges to the solution $\boldsymbol{x}^*$ of the linear system and minimization problem in at most n steps.

Proof:

$$\min_{\alpha} f(\boldsymbol{x}_k + \alpha \boldsymbol{d}_k)$$

We can compute the derivative with respect to α:

$$\begin{aligned}\frac{\partial}{\partial \alpha} f(\boldsymbol{x} + \alpha \boldsymbol{d}) &= \frac{\partial}{\partial \alpha}(\boldsymbol{x} + \alpha \boldsymbol{d})^T A(\boldsymbol{x} + \alpha \boldsymbol{d}) - \boldsymbol{b}^T(\boldsymbol{x} + \alpha \boldsymbol{d}) \\ &= \boldsymbol{d}^T A(\boldsymbol{x} + \alpha \boldsymbol{d}) - \boldsymbol{d}^T \boldsymbol{b} \\ &= \boldsymbol{d}^T(A\boldsymbol{x} - \boldsymbol{b}) + \alpha \boldsymbol{d}^T A \boldsymbol{d}\end{aligned}$$

Setting $\frac{\partial f(\boldsymbol{x}+\alpha \boldsymbol{d})}{\partial \alpha} = 0$ results in:

$$\alpha_k = -\frac{\boldsymbol{d}_k^T(A\boldsymbol{x}_k - \boldsymbol{b})}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} = -\frac{\boldsymbol{d}_k^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} \tag{1}$$

- Since the directions $\{\boldsymbol{d}_k\}$ are linearly independent, they must span the whole space $\mathbb{R}^n$. Hence, there is a set of scalars σ_k such that:

$$\boldsymbol{x}^* - \boldsymbol{x}_0 = \sigma_0 \boldsymbol{d}_0 + \sigma_1 \boldsymbol{d}_1 + \ldots + \sigma_{n-1} \boldsymbol{d}_{n-1} \qquad \text{(the "in at most } n \text{ steps" part)}$$

- By premultiplying this expression by $\boldsymbol{d}_k^T A$ and using the conjugacy property, we obtain:

$$\sigma_k = \frac{\boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_0)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k} \tag{2}$$

- If $\boldsymbol{x}_k$ is generated by conjugate direction algorithm, then we have

$$\boldsymbol{x}_k = \boldsymbol{x}_0 + \alpha_0 \boldsymbol{d}_0 + \alpha_1 \boldsymbol{d}_1 + \ldots + \alpha_k \boldsymbol{d}_{k-1}$$

- By premultiplying this expression by $\boldsymbol{d}_k^T A$ and using the conjugacy property, we have that

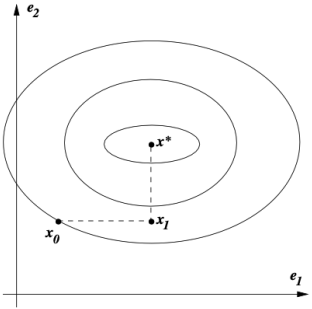
$$\boldsymbol{d}_k^T A(\boldsymbol{x}_k - \boldsymbol{x}_0) = 0$$

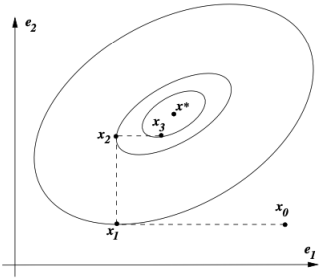
- and therefore

$$\begin{aligned} \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_0) &= \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k + \boldsymbol{x}_k - \boldsymbol{x}_0) = \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k) + \boldsymbol{d}_k^T A(\boldsymbol{x}_k - \boldsymbol{x}_0) = \\ &= \boldsymbol{d}_k^T A(\boldsymbol{x}^* - \boldsymbol{x}_k) = \boldsymbol{d}_k^T (\boldsymbol{b} - A\boldsymbol{x}_k) = -\boldsymbol{d}_k^T \boldsymbol{r}_k. \end{aligned}$$

- Using this result in (2) and comparing with (1) we conclude $\alpha_k = \sigma_k$. $\square$

Interpretation of Conjugate Directions





If the matrix A is diagonal, the contours of the function $f(\cdot)$ are ellipses whose axes are aligned with the coordinate directions. In each iteration we compute one component of the optimal solution, this takes two steps in $\mathbb{R}^2$.

If A is not diagonal, its contours are elliptical, but they are usually not aligned with the coordinate directions. Finding the optimum may take more than n iterations.

Transform the problem to make A diagonal and minimize along the coordinate directions.

Transformation matrix: $S = \left[\boldsymbol{d}_0, \boldsymbol{d}_1, \ldots, \boldsymbol{d}_{n-1}\right]$

Conjugate Gradient Method

- The **conjugate gradient method** is a **first order method** with the property: In generating its set of conjugate vectors, it can compute a new vector $\boldsymbol{d}_k$ by using only the previous vector $\boldsymbol{d}_{k-1}$. Hence, little storage and computation requirements.

$$\boxed{\boldsymbol{d}_k = -\boldsymbol{r}_k + \beta_k \boldsymbol{d}_{k-1}}$$

where β_k is to be determined such that $\boldsymbol{d}_{k-1}$ and $\boldsymbol{d}_k$ are conjugate with respect to A. By premultiplying by $\boldsymbol{d}_{k-1}^T A$ and imposing that $\boldsymbol{d}_{k-1}^T A \boldsymbol{d}_k = 0$ we find that

$$\beta_k = \frac{\boldsymbol{r}_k^T A \boldsymbol{d}_{k-1}}{\boldsymbol{d}_{k-1}^T A \boldsymbol{d}_{k-1}}$$

- Larger values of β indicate that the previous descent direction contributes more strongly.
- $\boldsymbol{d}_0$ is commonly chosen to be the steepest descent direction at $\boldsymbol{x}_0$
- Advantage with respect to steepest descent: implicitly reuses previous information about the function and thus better convergence.

Algorithm CG

Basic version:

Input: $f, \boldsymbol{x}_0$
Output: x^*
Set $\boldsymbol{r}_0 \leftarrow A\boldsymbol{x}_0 - b, \boldsymbol{d}_0 \leftarrow \boldsymbol{r}_0, k \leftarrow 0$;
while $\boldsymbol{r}_k \neq 0$ **do**

$$\alpha_k \leftarrow -\frac{\boldsymbol{d}_k^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k};$$
$$\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k;$$
$$\boldsymbol{r}_{k+1} \leftarrow A\boldsymbol{x}_{k+1} - \boldsymbol{b};$$
$$\beta_{k+1} \leftarrow \frac{\boldsymbol{r}_{k+1}^T A \boldsymbol{d}_k}{\boldsymbol{d}_k^T A \boldsymbol{d}_k};$$
$$\boldsymbol{d}_{k+1} \leftarrow -\boldsymbol{r}_{k+1} + \beta_{k+1} \boldsymbol{d}_k;$$
$$k \leftarrow k + 1;$$

Computationally improved version:

Input: $f, \boldsymbol{x}_0$
Output: x^*
Set $\boldsymbol{r}_0 \leftarrow A\boldsymbol{x}_0 - b, \boldsymbol{d}_0 \leftarrow \boldsymbol{r}_0, k \leftarrow 0$;
while $\boldsymbol{r}_k \neq 0$ **do**

$$\alpha_k \leftarrow -\frac{\boldsymbol{r}(\boldsymbol{x}_k)^T \boldsymbol{r}(\boldsymbol{x}_k)}{\boldsymbol{d}_k^T A \boldsymbol{d}_k};$$
$$\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k;$$
$$\boldsymbol{r}_{k+1} \leftarrow \boldsymbol{r}_k + \alpha_k A \boldsymbol{d}_k;$$
$$\beta_{k+1} \leftarrow \frac{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^T \boldsymbol{r}_k};$$
$$\boldsymbol{d}_{k+1} \leftarrow -\boldsymbol{r}_{k+1} + \beta_{k+1} \boldsymbol{d}_k;$$
$$k \leftarrow k + 1;$$

- we never need to know the vectors $\boldsymbol{x}, \boldsymbol{r}$, and $\boldsymbol{d}$ for more than the last two iterations.
- major computational tasks: the matrix–vector product $A\boldsymbol{d}_k$, inner products $\boldsymbol{d}_k^T A \boldsymbol{d}_k$ and $\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}$, and three vector sums

NonLinear Conjugate Gradient Methods

- The conjugate gradient method can be applied to nonquadratic functions as well.
- Smooth, continuous functions behave like quadratic functions close to a local minimum
- but! we do not know the value of A that best approximates f around $\boldsymbol{x}_k$. Instead, several choices for β_k tend to work well
- Two changes:
  - α_k is computed by solving an approximate line search
  - the residual $\boldsymbol{r}$, (it was simply the gradient of f), must be replaced by the gradient of the nonlinear objective f.

NonLinear Conjugate Gradient Methods

Fletcher-Reeves Method:

Input: $f, \boldsymbol{x}_0$
Output: x^*
Evaluate $f_0 = f(\boldsymbol{x}_0), \nabla f_0 = \nabla f(\boldsymbol{x}_0)$;
Set $\boldsymbol{d}_0 \leftarrow -\nabla f_0, k \leftarrow 0$;
while $\nabla f_k \neq 0$ **do**
  Compute α_k by line search and set $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
  Evaluate ∇f_{k+1};
  $\beta_{k+1}^{FR} \leftarrow \frac{\nabla f_{k+1}^T \nabla f_{k+1}}{\nabla f_k^T \nabla f_k}$;
  $\boldsymbol{d}_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{FR} \boldsymbol{d}_k$;
  $k \leftarrow k + 1$;

Polak-Ribière:

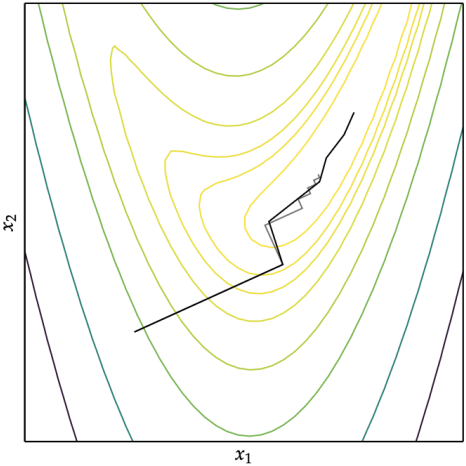
Input: $f, \boldsymbol{x}_0$
Output: x^*
Evaluate $f_0 = f(\boldsymbol{x}_0), \nabla f_0 = \nabla f(\boldsymbol{x}_0)$;
Set $\boldsymbol{d}_0 \leftarrow -\nabla f_0, k \leftarrow 0$;
while $\nabla f_k \neq 0$ **do**
  Compute α_k by line search and set $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$;
  Evaluate ∇f_{k+1};
  $\beta_{k+1}^{PR} \leftarrow \frac{\nabla f_{k+1}^T (\nabla f_{k+1} - \nabla f_k)}{\nabla f_k^T \nabla f_k}$;
  $\boldsymbol{d}_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{PR} \boldsymbol{d}_k$;
  $k \leftarrow k + 1$;

PR with:

$$\beta_{k+1}^{+} = \max\{\beta_{k+1}^{PR}, 0\}$$

becomes PR$^+$ and guaranteed to converge (satisfies first Wolfe conditions).

The conjugate gradient method with the Polak-Ribière update. Gradient descent is shown in gray.



Outline

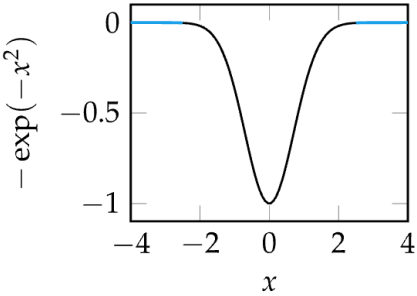
10. Gradient Descent

11. Conjugate Descent

12. Accelerated Descents

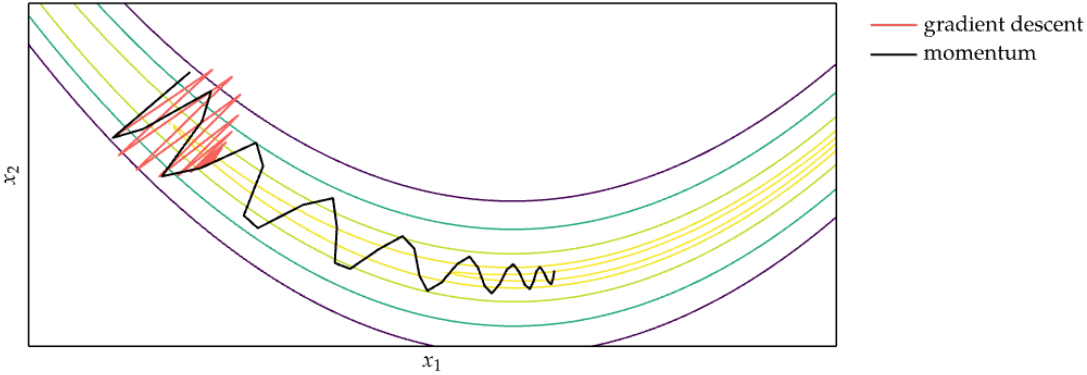
Accelerated Descents

- Addresses common convergence issues
- Some functions have regions with very small gradients (flat surface) where gradient descent gets stuck



Momentum

Rosenbrock function with $b = 100$



Momentum overcomes these issues by replicating the effect of physical momentum

Momentum

Momentum update equations:

$$\boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k - \alpha \nabla f(\boldsymbol{x}_k)$$
$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{v}_{k+1}$$

```
import numpy as np

class Momentum(DescentMethod):
  alpha: float # learning rate
  beta: float # momentum decay
  v: np.array # momentum

  def __init__(self, alpha, beta, f, grad, x):
    self.alpha = alpha
    self.beta = beta
    self.v = np.zeros_like(x)

  def step(self, grad, x):
    self.v = self.beta * self.v - self.alpha * grad(x↪
        ↪)
    return x + self.v
```

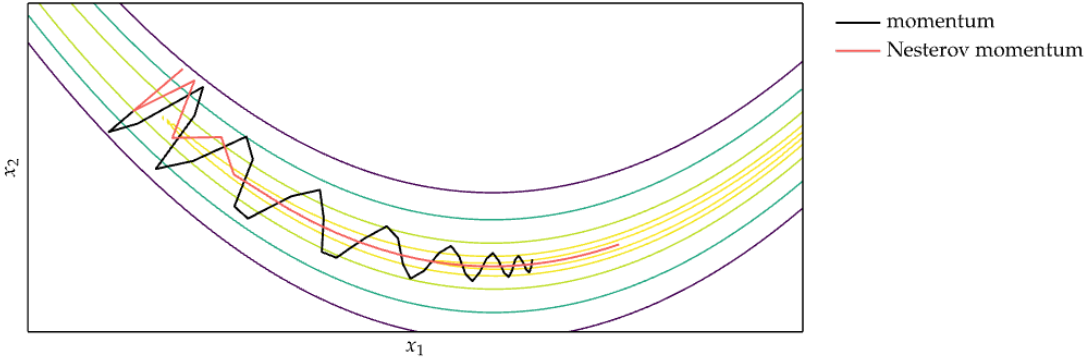
Nesterov Momentum Gradient (NAG)

Issue of momentum: steps do not slow down enough at the bottom of a valley, overshoot.

Nesterov Momentum update equations:

$$\boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k - \alpha \nabla f(\boldsymbol{x}_k + \beta \boldsymbol{v}_k)$$
$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{v}_{k+1}$$

it calculates the gradient at a "look-ahead" position



Adagrad

- Instead of using the same learning rate for all components of $\boldsymbol{x}$,
  Adaptive Subgradient method (Adagrad) adapts the learning rate for each component of $\boldsymbol{x}$.
  For each component of $\boldsymbol{x}$, the update equation is

$$x_{i,k+1} = x_{i,k} - \frac{\alpha}{\epsilon + \sqrt{s_{i,k}}} \nabla f_i(\boldsymbol{x}_k)$$

  where

$$s_{i,k} = \sum_{j=1}^{k} (\nabla f_i(\boldsymbol{x}_j))^2$$

$$\epsilon \approx 1 \times 10^{-8}, \alpha = 0.01$$

- components of $\boldsymbol{s}$ are strictly nondecreasing, hence learning rate decreases over time

RMSProp

- Extends Adagrad to avoid monotonically decreasing learning rate by maintaining a decaying average of squared gradients

$$\hat{s}_{k+1} = \gamma \hat{s}_k + (1-\gamma)\left(\nabla f(\boldsymbol{x}_k) \odot \nabla f(\boldsymbol{x}_k)\right), \qquad \gamma \in [0,1], \qquad \odot \text{ element-wise product}$$

(accumulated squared gradients) and using it in the Update Equation

$$\begin{aligned} x_{i,k+1} &= x_{i,k} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_{i,k}}} \nabla f_i(\boldsymbol{x}_k) \\ &\approx x_{i,k} - \frac{\alpha}{\epsilon + RMS(\nabla f_i(\boldsymbol{x}_k))} \nabla f_i(\boldsymbol{x}_k) \end{aligned}$$

root mean square: For n values $\{x_1, x_2, \ldots, x_n\}$

$$x_{\text{RMS}} = \sqrt{\frac{1}{n}\left({x_1}^2 + {x_2}^2 + \cdots + {x_n}^2\right)}.$$

AdaDelta

Also extends Adagrad to avoid monotonically decreasing learning rate
Modifies RMSProp to eliminate learning rate parameter entirely

$$x_{i,k+1} = x_{i,k} - \frac{RMS(\Delta x_i)}{\epsilon + RMS(\nabla f_i(\boldsymbol{x}))} \nabla f_i(\boldsymbol{x}_k)$$

Δx_i sum of squared updates
$\nabla f_i(\boldsymbol{x})$ sum of squared gradients

Adam

- The **adaptive moment estimation method** (Adam), adapts the learning rate to each parameter.
- stores both an exponentially decaying gradient like momentum and an exponentially decaying squared gradient like RMSProp and Adadelta
- At each iteration, a sequence of values are computed

$$
\begin{aligned}
\text{Biased decaying momentum} \quad & \boldsymbol{v}_{k+1} = \beta \boldsymbol{v}_k - \alpha \nabla f(\boldsymbol{x}_k) \\
\text{Biased decaying squared gradient} \quad & \boldsymbol{s}_{k+1} = \gamma \boldsymbol{s}_k + (1-\gamma)\left(\nabla f(\boldsymbol{x}_k) \odot \nabla f(\boldsymbol{x}_k)\right) \\
\text{Corrected decaying momentum} \quad & \hat{\boldsymbol{v}}_{k+1} = \boldsymbol{v}_{k+1}/(1-\gamma_{v,k}) \\
\text{Corrected decaying squared gradient} \quad & \hat{\boldsymbol{s}}_{k+1} = \boldsymbol{s}_{k+1}/(1-\gamma_{s,k}) \\
\text{Next iterate} \quad & \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{\alpha}{(\epsilon + \sqrt{\hat{\boldsymbol{s}}_{k+1}})} \hat{\boldsymbol{v}}_{k+1}
\end{aligned}
$$

- Defaults: $\alpha = 0.001, \gamma_v = 0.9, \gamma_s = 0.999, \epsilon = 1 \times 10^{-8}$

Adamax

Same as Adam, but based on the max-norm L_∞.

$$\begin{aligned} \boldsymbol{s}_{k+1} &= \gamma^\infty \boldsymbol{s}_k + (1-\gamma^\infty)\left(\|\nabla f(\boldsymbol{x}_k)\|_\infty\right) \\ &= \max\left(\gamma \boldsymbol{s}_k, \|\nabla f(\boldsymbol{x}_k)\|_\infty\right) \end{aligned}$$

Nadam

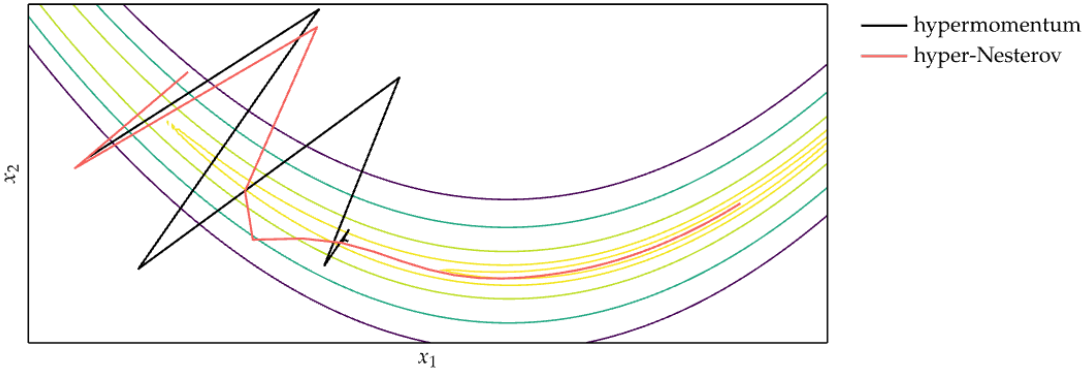
Nadam

- Nesterov-accelerated Adaptive Moment Estimation
- Adam is basically RMSProp with momentum
- We have seen that Nesterov is often more efficient
- Welcome to Nadam: Adam which uses the Nesterov momentum.

Hypergradient Descent

- Learning rate determines how sensitive the method is to the gradient signal.
- Many accelerated descent methods are highly sensitive to hyperparameters such as learning rate.
- Applying gradient descent to a hyperparameter of an underlying descent method is called hypergradient descent
- Requires computing the partial derivative of the objective function with respect to the hyperparameter

Hypergradient Descent



Summary

- Gradient descent follows the direction of steepest descent.
- The conjugate gradient method can automatically adjust to local valleys.
- Descent methods with momentum build up progress in favorable directions.
- A wide variety of accelerated descent methods use special techniques to speed up descent.
- Hypergradient descent applies gradient descent to the learning rate of an underlying descent method.

6. Second-Order Methods

Descent Direction Methods

How to select the descent direction?

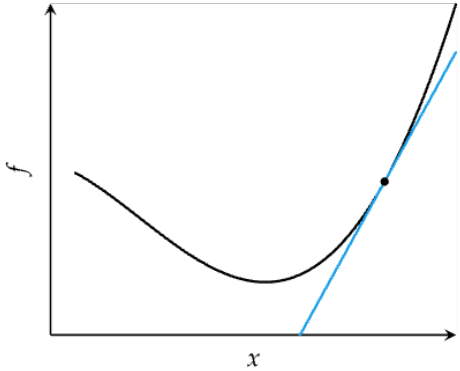
- first-order methods that rely on gradient
- second-order methods that rely on Hessian information

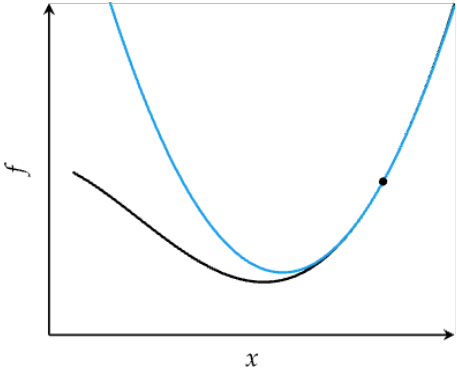
Advantages of second order methods in descent algorithms:

- way of accelerating the iteration [Davidon mid 1950s]
- additional information that can help improve the local model for informing the selection of
  - directions and
  - step lengths

Second-Order Methods

- Locally approximate function as quadratic
- Comparison of first-order and second order approximations





Outline

13. Newton Method

14. Secant Method

15. Quasi-Newton Method

Newton's Method – Univariate

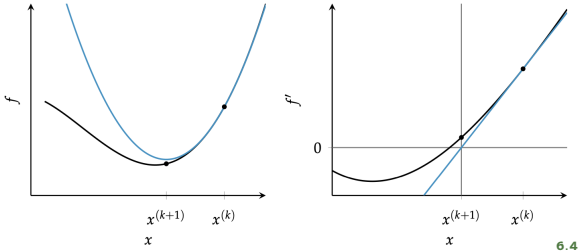
- Approximate a function using second-order Taylor series expansion
- analytically obtain the location where a quadratic approximation has a zero gradient.
- use that location as the next iteration to approach a local minimum.
- Univariate function

$$q(x) = f(x_k) + (x - x_k)f'(x_k) + \frac{(x - x_k)^2}{2} f''(x_k)$$

$$\frac{\mathrm{d}q(x)}{\mathrm{d}x} = f'(x_k) + (x - x_k)f''(x_k) = 0$$

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

$\equiv$ finding roots of derivative function



Newton's Method - Multivariate

- Multivariate function

$$f(\boldsymbol{x}) \approx q(\boldsymbol{x}) = f(\boldsymbol{x}_k) + \nabla f(\boldsymbol{x}_k)^T(\boldsymbol{x} - \boldsymbol{x}_k) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}_k)^T H_k(\boldsymbol{x} - \boldsymbol{x}_k)$$

- H is the Hessian matrix
- Evaluate the gradient and set it to zero:

$$\nabla \boldsymbol{q}(x) = \nabla f(\boldsymbol{x}_k) + H(\boldsymbol{x}_k)(\boldsymbol{x} - \boldsymbol{x}_k) = 0$$

- Multivariate update rule

$$\boxed{\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - H_k^{-1}\nabla f(\boldsymbol{x}_k)}$$

- (If f is quadratic and its Hessian is positive definite, then the update converges to the global minimum in one step.)

Algorithm

Input: $\nabla f, H, \boldsymbol{x}_0, \epsilon, k_{max}$
Output: $\boldsymbol{x}^*$
Set $k = 0, \Delta = 1, \boldsymbol{x} = \boldsymbol{x}_0$;
while $\|\Delta\| > \epsilon$ and $k \leq k_{max}$ **do**
  $\Delta = H(\boldsymbol{x})^{-1}\nabla f(\boldsymbol{x})$;
  $\boldsymbol{x} = \boldsymbol{x} - \Delta$;
  $k = k + 1$;

It can be modified to only give a descent direction $\boldsymbol{d} = -H(\boldsymbol{x})^{-1}\nabla f(\boldsymbol{x})$ and leave the step size to be determined with line search.

Newton's method – Example

Minimize **Booth's function**:

$$f(x) = (x_1 + 2x_2 - 7)^2 + (2x_1 + x_2 - 5)^2$$

- $\boldsymbol{x}_0 = [9, 8]$
- The gradient of Booth's function is:

$$\nabla f(x) = [10x_1 + 8x_2 - 34, 8x_1 + 10x_2 - 38]$$

- The Hessian of Booth's function is:

$$H(x) = \begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}$$

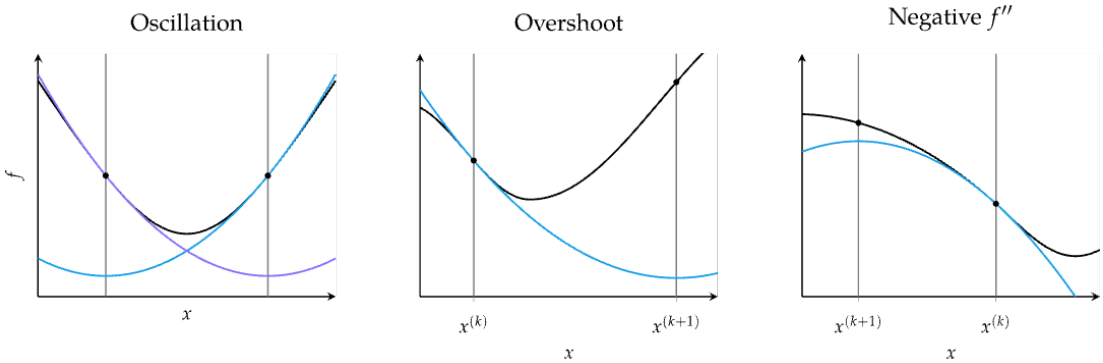
- The first iteration of Newton's method yields:

$$\begin{aligned}\boldsymbol{x}_1 &= \boldsymbol{x}_0 - H(\boldsymbol{x}_0)^{-1}\nabla f(\boldsymbol{x}_0)\\ &= \begin{bmatrix}9\\8\end{bmatrix} - \begin{bmatrix}10 & 8\\8 & 10\end{bmatrix}^{-1}\cdot\begin{bmatrix}10\cdot 9 + 8\cdot 8 - 34\\8\cdot 9 + 10\cdot 8 - 38\end{bmatrix} = \begin{bmatrix}9\\8\end{bmatrix} - \begin{bmatrix}10 & 8\\8 & 10\end{bmatrix}^{-1}\cdot\begin{bmatrix}120\\114\end{bmatrix} = \begin{bmatrix}1\\3\end{bmatrix}\end{aligned}$$

- Second iteration: The gradient at $\boldsymbol{x}_1$ is zero, so we have converged after a single iteration. The Hessian is positive definite everywhere, so $\boldsymbol{x}_1$ is the global minimum.

Newton's Method

Common causes of error in Newton's method



Strong Convexity

Recall that for a symmetric matrix A, $A \succeq 0$ means that A is positive semidefinite, i.e.

$$\boldsymbol{x}^\top A\boldsymbol{x} \geq 0 \quad \forall \boldsymbol{x} \in \mathbb{R}^n.$$

Hence, for a Q symmetric matrix, and I the identity matrix, the condition

$$Q \succeq cI \qquad \text{means:} \qquad Q - cI \succeq 0.$$

In scalar form:

$$\boldsymbol{x}^T(Q - cI)\boldsymbol{x} = \boldsymbol{x}^T Q\boldsymbol{x} - c\|\boldsymbol{x}\|^2$$

$$\boldsymbol{x}^\top Q\boldsymbol{x} \geq c\|\boldsymbol{x}\|^2 \quad \forall \boldsymbol{x}.$$

this was explained wrongly in class
and it has been corrected here

Strong Convexity

Eigenvalue Interpretation:

Because Q is symmetric, it has an orthonormal eigenbasis.
Let its eigenvalues be $\lambda_1, \ldots, \lambda_n$.
Then:

$$Q \succeq cI \iff \lambda_i \geq c \quad \forall i.$$

In other words:

$$\lambda_{\min}(Q) \geq c.$$

So the smallest curvature direction of Q is at least c.

If

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 100 \end{bmatrix},$$

then eigenvalues are 1 and 100. So:
So:

- $Q \succeq I$
- $Q \not\succeq 2I$ (since $1 < 2$)

The smallest curvature direction determines the strong convexity constant.

Strong Convexity

Geometric Meaning:

Consider the quadratic function

$$f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top Q\boldsymbol{x}.$$

Its curvature in direction $\boldsymbol{v}$ is $\boldsymbol{v}^\top Q\boldsymbol{v}$.
The condition $Q \succeq cI$ means:

$$\boldsymbol{v}^\top Q\boldsymbol{v} \geq c\|\boldsymbol{v}\|^2 \quad \forall \boldsymbol{v}.$$

So:
The function curves upward at least as much as a bowl with curvature c in every direction.

Strong Convexity

Definition (Strongly Convex Function (Lower Curvature Bound))

The function $f : \mathbb{R}^d \to \mathbb{R}$ twice-differentiable function (ie, $f \in C^2(\mathbb{R}^n)$) is c-**strongly convex** if there exists a constant $c > 0$ such that for all $\boldsymbol{x}$ and $\boldsymbol{y}$ in $\mathbb{R}^d$:

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}) + \frac{c}{2}\|\boldsymbol{y} - \boldsymbol{x}\|^2.$$

or, equivalently, if there exists $c > 0$:

$$\nabla^2 f(\boldsymbol{x}) \succeq cI \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^d$$

(for univariate case: $f''(x) \geq c$), ie, grows at least quadratically.

When $c = 0$, we recover the basic inequality characterizing convexity; for $c > 0$ we obtain a better lower bound on $f(\boldsymbol{x})$ than follows from convexity alone.

Convex $\to$ no flat "bending downward"; Strongly convex $\to$ strictly curved upward everywhere

Hence, f has a unique minimizer, denoted as $\boldsymbol{x}^* \in \mathbb{R}^d$ with $f^* \stackrel{def}{=} f(\boldsymbol{x}^*)$.

Strong Convexity

Interpretation

Compared to ordinary convexity:

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})^\top (\boldsymbol{y} - \boldsymbol{x}),$$

strong convexity adds a quadratic curvature term.
It enforces that the function grows at least quadratically away from any point.
Equivalently:

$$f(\boldsymbol{x}) - \frac{c}{2}\|\boldsymbol{x}\|^2 \text{ is convex.}$$

Monotonicity of Gradient
Strong convexity is equivalent to strong monotonicity of the gradient:

$$(\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y}))^\top (\boldsymbol{x} - \boldsymbol{y}) \geq c\|\boldsymbol{x} - \boldsymbol{y}\|^2 \quad \forall \boldsymbol{x}, \boldsymbol{y}.$$

This is extremely important in convergence proofs.

Strong Convexity and Lipschitz Gradient

Definition (Lipschitz Gradient (Upper Curvature Bound))

f has an L-Lipschitz gradient if

$$\|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|.$$

Equivalent (if $f \in C^2(\mathbb{R}^n)$):

$$\nabla^2 f(\boldsymbol{x}) \preceq LI \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^n$$

This gives a uniform upper bound on curvature.

<u>Combined Condition</u>:
If both hold:

$$\mu I \preceq \nabla^2 f(\boldsymbol{x}) \preceq LI \qquad \text{for all } \boldsymbol{x} \in \mathbb{R}^n$$

then all eigenvalues of the Hessian satisfy:

$$\mu \leq \lambda_i(\nabla^2 f(\boldsymbol{x})) \leq L.$$

So:
Strong convexity $\rightarrow$ prevents flatness
Lipschitz gradient $\rightarrow$ prevents excessive curvature

Newton's Method - Multivariate

Theorem

If f is a strongly convex function with Lipschitz continuous Hessian then provided that x_0 is close enough to $x_* = \arg\min f(x)$, the sequence $x_0, x_1, x_2, \ldots$ generated by Newton's method will converge to the (necessarily unique) minimizer x_* of f quadratically fast. That is,

$$\|x_{k+1} - x_*\| \leq \frac{1}{2}\|x_k - x_*\|^2, \qquad \forall k \geq 0.$$

Lipschitz Hessian is a third order information on the variation of curvature.
Strong convexity says: The bowl never flattens. Lipschitz Hessian says: The bowl shape cannot change too abruptly.

Newton's Method

- has quadratic convergence, meaning the difference between the minimum and the iterate is approximately squared with every iteration.

- This rate of convergence holds for Newton's method starting from x_0 within an interval $I = [x^* - \delta, x^* + \delta]$, for a root x^*, if

  1. $f''(x) \neq 0$ for all points in I,

  2. $f'''(x)$ is continuous on I, and

  3. $\frac{1}{2}\left|\frac{f'''(x_0)}{f''(x_0)}\right| < c\left|\frac{f'''(x^*)}{f''(x^*)}\right|$ for some $c < \infty$
     sufficient closeness condition, ensuring that the function is sufficiently approximated by the Taylor expansion and no overshoot.

Outline

13. Newton Method

14. Secant Method

15. Quasi-Newton Method

Secant Method – Univariate

- For univariate functions, if the second derivative is unknown, it can be approximated using the secant method

$$f''(x_k) = \frac{f'(x_k) - f'(x_{k-1})}{x_k - x_{k-1}}$$

- Update equation

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f'(x_k) - f'(x_{k-1})} f'(x_k)$$

- It requires an additional initial design point and suffers from the same problems as Newton's method and may take more iterations to converge due to approximating the second derivative.

Outline

13. Newton Method

14. Secant Method

15. Quasi-Newton Method

Quasi-Netwon Methods – Multivariate

- Automatic differentiation tools may not be applicable in many situations, and it may be much more costly to work with second derivatives in automatic differentiation software than with the gradient.
- Quasi-Newton methods, like steepest descent, require only the gradient of the objective function to be supplied at each iterate.
- By measuring the changes in gradients, they construct a model of the objective function that is good enough to produce superlinear convergence.
- The improvement over steepest descent is dramatic, especially on difficult problems.

Quasi-Netwon Methods – Multivariate

Use an approximation $Q_k \approx H^{-1}(\boldsymbol{x}_k)$

Input: $\boldsymbol{x}_0$, convergence tolerance $\epsilon > 0$, Q_0 (typically the $n \times n$ identity matrix)
Output: x^*
Set $k \leftarrow 0$;
while $\|\nabla f(\boldsymbol{x}_k)\| > \epsilon$ **do**
  Compute search direction $\boldsymbol{d}(x_k) = -Q_k \nabla f(\boldsymbol{x}_k)$;
  Set $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{d}(\boldsymbol{x}_k)$ where α_k is computed from a line search procedure to satisfy the Wolfe conditions;
  Define $\boldsymbol{\delta}_{k+1} \stackrel{def}{=} \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$ and $\boldsymbol{\gamma}_{k+1} \stackrel{def}{=} \nabla f(\boldsymbol{x}_{k+1}) - \nabla f(\boldsymbol{x}_k)$;
  Compute Q_{k+1};
  $k \leftarrow k + 1$;

- Davidon-Fletcher-Powell (DFP) method
- Broyden-Fletcher-Goldfarb-Shanno (BFGS) method
- Limited-memory BFGS (L-BFGS) method

Davidon-Fletcher-Powell (DFP) method

$$Q_{k+1} = Q_k - \frac{Q_k \gamma_k \gamma_k^T Q_k}{\gamma_k^T Q_k \gamma_k} + \frac{\boldsymbol{\delta}_k \boldsymbol{\delta}_k^T}{\boldsymbol{\delta}_k^T \gamma_k}$$

where all terms on the right hand side are evaluated at the same iteration k.

The update for Q in the DFP method has three properties:

- Q remains symmetric and positive definite.
- If $f(x) = \frac{1}{2}\boldsymbol{x}^T A\boldsymbol{x} + \boldsymbol{b}^T\boldsymbol{x} + c$, then $Q = A^{-1}$. Thus the DFP has the same convergence properties as the conjugate gradient method.
- For high-dimensional problems, storing and updating Q can be significant compared to other methods like the conjugate gradient method.

Broyden-Fletcher-Goldfarb-Shanno (BFGS) method

$$Q_{k+1} = Q_k - \left(\frac{\boldsymbol{\delta}_k \boldsymbol{\gamma}_k^T Q_k + Q_k \boldsymbol{\gamma}_k \boldsymbol{\delta}_k^T}{\boldsymbol{\delta}_k^T \boldsymbol{\gamma}_k} \right) + \left(1 + \frac{\boldsymbol{\gamma}_k^T Q_k \boldsymbol{\gamma}_k}{\boldsymbol{\delta}_k^T \boldsymbol{\gamma}_k} \right) \frac{\boldsymbol{\delta}_k \boldsymbol{\delta}_k^T}{\boldsymbol{\delta}_k^T \boldsymbol{\gamma}_k}$$

BFGS better than DFP with approximate line search but still uses an $n \times n$ dense matrix.

<u>Theorem</u>: Suppose that f is twice continuously differentiable and that the iterates generated by the BFGS algorithm converge to a minimizer $\boldsymbol{x}^*$ at which the Hessian matrix G is Lipschitz continuous Suppose also that the sequence $\|\boldsymbol{x}_k - \boldsymbol{x}^*\|$ converges to zero rapidly enough that $\sum_{k=1}^{\infty} \|\boldsymbol{x}_k - \boldsymbol{x}^*\| < \infty$. Then $\boldsymbol{x}_k$ converges to $\boldsymbol{x}^*$ at a superlinear rate (ie, faster than linear).

Limited-memory BFGS (L-BFGS) method

For **large-scale unconstrained optimization**
It stores the last m values for δ and γ rather than the full inverse Hessian ($i = 1$ oldest, $i = m$ last).
Compute $\boldsymbol{d}$ at $\boldsymbol{x}$ as $\boldsymbol{d} = -\boldsymbol{z}_m$ using:

$$\boldsymbol{q}_m = \nabla f(\boldsymbol{x}_k) \qquad \boldsymbol{q}_i = \boldsymbol{q}_{i+1} - \frac{\boldsymbol{\delta}_{i+1}^T \boldsymbol{q}_{i+1}}{\boldsymbol{\gamma}_{i+1}^T \boldsymbol{\delta}_{i+1}} \boldsymbol{\gamma}_{i+1}, \quad i = m-1, \ldots, 1$$

$$\boldsymbol{z}_0 = \frac{\boldsymbol{\delta}_m \odot \boldsymbol{\delta}_m \odot \boldsymbol{q}_m}{\boldsymbol{\gamma}_m^T \boldsymbol{\gamma}_m} \qquad \boldsymbol{z}_i = \boldsymbol{z}_{i-1} + \boldsymbol{\delta}_{i-1} \left(\frac{\boldsymbol{\delta}_{i-1}^T \boldsymbol{q}_{i-1}}{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{\delta}_{i-1}} - \frac{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{z}_{i-1}}{\boldsymbol{\gamma}_{i-1}^T \boldsymbol{\gamma}_{i-1}} \right), \quad i = 1, \ldots, m$$

For minimization, the inverse Hessian Q must remain positive definite.
The initial Hessian is often set to the diagonal of

$$Q_0 = \frac{\boldsymbol{\gamma}_0 \boldsymbol{\delta}_0^T}{\boldsymbol{\gamma}_0^T \boldsymbol{\gamma}_0}$$

Computing the diagonal for the above expression and substituting the result into $\boldsymbol{z}_0 = Q_0 \boldsymbol{q}_0$ results in the equation for $\boldsymbol{z}_0$.

L-BFGS

(L-BFGS two-loop recursion)

Input:

Output: $Q_k \nabla f_k = \boldsymbol{z}$

Set $\boldsymbol{q} \leftarrow \nabla f_k$;

for $i = k-1, k-2, \ldots, k-m$ **do**

  $\alpha_i \leftarrow \frac{\delta_i^T \boldsymbol{q}}{\gamma_i \cdot \boldsymbol{\delta}_i^T}$;

  $\boldsymbol{q} \leftarrow \boldsymbol{q} - \alpha_i \cdot \gamma_i$;

$\boldsymbol{z} \leftarrow Q_0 \boldsymbol{q}$;

for $i = k-m, k-m+1, \ldots, k-1$ **do**

  $\beta \leftarrow \frac{\gamma_i^T \cdot \boldsymbol{r}}{\gamma_i \cdot \boldsymbol{\delta}_i^T}$;

  $\boldsymbol{z} \leftarrow \boldsymbol{z} + \delta_i(\boldsymbol{z}_i - \beta)$;

Input:

Output: x^*

Choose starting point $\boldsymbol{x}_0$, integer $m > 0$;

$k \leftarrow 0$;

while not convergence **do**

  Set Q_0;

  Compute $\boldsymbol{d}_k \leftarrow -Q_k \nabla f_k$ from Algorithm on the left;

  Compute $\boldsymbol{x}_{k+1} \leftarrow \boldsymbol{x}_k + \alpha_k \boldsymbol{d}_k$, where α_k is chosen to satisfy the Wolfe conditions;

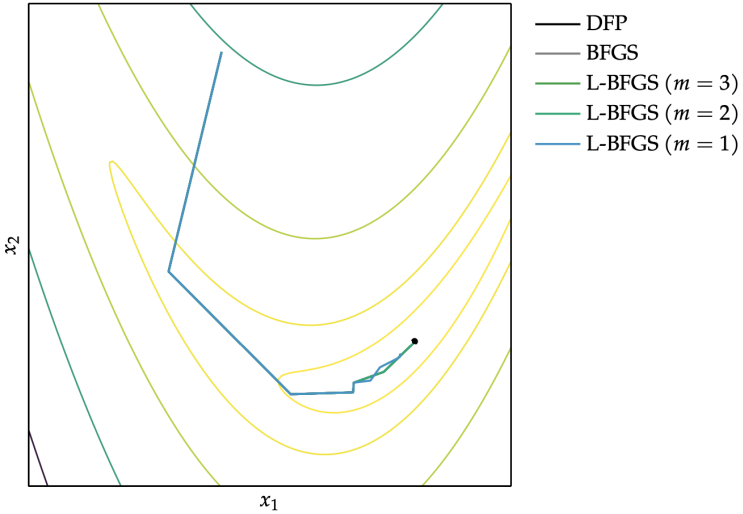
  if $k > m$ **then**

    Discard the vector pair $\{\boldsymbol{\delta}_{k-m}, \boldsymbol{\gamma}_{k-m}\}$ from storage;

  Compute and save $\boldsymbol{\delta}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, $\gamma_k = \nabla f_{k+1} - \nabla f_k$;

  $k \leftarrow k+1$;

BFGS Methods - Comparison



Summary

- Incorporating second-order information in descent methods often speeds convergence.
- Newton's method is a root-finding method that leverages second-order information to quickly descend to a local minimum.
- The secant method and quasi-Newton methods approximate Newton's method when the second-order information is not directly available.
- In Python, methods implemented in the module scipy
  https://docs.scipy.org/doc/scipy/tutorial/optimize.html